Advanced User Interfaces - Politecnico di Milano
Project Documentation

3.6 - LLM in Fabula

Group 05
Stefano Riva, Diego Tagliabue, Francesca Trevisan

A.A. 2025-2026

Abstract

LLM in Fabula is a new version of the famous party game Lupus in Fabula in which players in the same place can use their phones to make the game easier.
In this game, the key element to enjoy a session is having a good narrator leading the story and the flow of the game. *LLM in Fabula* provide a LLM narrator that helps the players fully enjoy the game while it leads the session in a more balanced and engaging way. Developed has a mobile application, it makes the game accessible and enjoyable for people of all ages, as children do not need an adult to help them understanding the rules or narrating the story, since the LLM does it instead. Each phase of the session is managed by the Artificial Intelligence, that speaks from the host phone, showing how the use of this new technology can actually help connecting people sometimes.

The Team

STEFANO RIVA
Computer Science student enrolled in the second year of Computer Science and Engineering, coming from Computer science, in which he graduated in 2024 with a Bachelor Degree from Politecnico di Milano.

DIEGO TAGLIABUE
Engineering student currently attending the first year of Computer Science and Engineering, having finished Computer science, where he obtained a Bachelor Degree in 2025 at Politecnico di Milano. He is now continuing his academic path.

FRANCESCA TREVISAN
Design student enrolled in the first year of Digital Interaction Design MSc, coming from the field of Interior Design, in which she graduated in 2025 with a Bachelor Degree from Politecnico di Milano.

Member's Contribution

Team member	Contribution
Stefano Riva	Backend development, documentation chapters 3 & 5.
Diego Tagliabue	Frontend development, documentation chapter 7.
Francesca Trevisan	Mockup, UX app design, video and presentation design, documentation chapters 1, 2, 4, 6

Table 1: Contribution table

Contents

1 Executive Summary **7**

2 Requirements **8**
2.1 UNG Diagram . . . 8
2.2 Stakeholders . . . 8
2.3 Needs of the Stakeholders . . . 9
2.4 Context . . . 10
2.5 Constraints . . . 13
2.6 Goals . . . 13
2.6.1 Stakeholders' . . . 13
2.6.2 Technological . . . 14

3 State of the art **16**
3.1 Games and LLMs: A short introduction . . . 16
3.1.1 The role of the Game Master . . . 16
3.2 LLMs as narrators . . . 17
3.2.1 Interactive digital narratives . . . 18
3.2.2 Nova . . . 18
3.2.3 LLM narrators in Dungeon&Dragons . . . 19
3.3 Lupus in Fabula evaluation framework . . . 20
3.3.1 Agent-Based framework in Unity . . . 20
3.3.2 Werewolf Arena . . . 21

4 Solution - UX Design **22**
4.1 General approach . . . 22
4.2 Interaction and workflow . . . 22
4.2.1 Pre-Setting . . . 24
4.2.2 Game-Setting . . . 24
4.2.3 Game Session . . . 26
4.2.4 End of the game . . . 27
4.3 User scenario . . . 27
4.4 Conversational interaction . . . 31

5 Implementation **32**
5.1 Overview . . . 32
5.2 Front-end . . . 32
5.2.1 User Flow and Main Features . . . 33
5.2.2 Architectural Implementation . . . 33
5.3 Back-end . . . 35
5.3.1 Server . . . 35
5.3.2 Redis . . . 37
5.3.3 Timer worker . . . 37

5.3.4 Database 38
5.3.5 LLM agent 39
5.3.6 Deployment 39
5.4 Use Cases 39
5.4.1 Create lobby 40
5.4.2 Join lobby 41
5.4.3 Start game 42
5.4.4 Night phase 44
5.4.5 Check win 45
5.4.6 Discussion 46

6 Value Proposition **47**

7 Discussion and Future Works **49**
7.1 Critical Reflection 49
7.2 Future Developments 50
7.3 Conclusion 51

Acronyms

- **AI**: Artificial Intelligence.
- **API**: Application Programming Interface.
- **DB**: DataBase.
- **DBMS**: DataBase Management System.
- **DM**: Dungeon Master.
- **DnD**: Dungeons&Dragons.
- **FSM**: Finite State Machine.
- **GM**: Game Master.
- **IDN**: Interactive Digital Narratives.
- **LLM**: Large Language Model.
- **NPC**: Non-Player Character.
- **ORM**: Object-Relational Mapping.
- **RAG**: Retrieval-Augmented Generation.
- **Redis**: REmote DIctionary Server.
- **RPG**: Role-Play Game.
- **SLM**: Small Language Model.
- **STT**: Speech To Text.
- **TTRPG**: TableTop Role-Play Game.
- **TTS**: Text To Speech.
- **UGC**: User Generated Content.
- **UI**: User Interface.
- **UNG Diagram**: User Needs Goal Diagram.
- **UX**: User Experience.
- **VDM**: Virtual Dungeon Master.
- **WS**: WebSocket.

1 Executive Summary

Nowadays, digital tools are reshaping the way people interact and connect with one another. This concerns also how they play and have fun together. If few years ago was strictly necessary to meet in order to play a game together, today everyone can easily connect with friends to do that, using a computer or even just a smartphone. Indeed, several applications were released in these past years to replace classical board games and help people play remotely.

However, people are starting to rediscover the authenticity of sharing these experiences physically together and not only behind a screen, even if traditional social games often face practical limitations.
Lupus in Fabula, a game built on narration, deception and group dynamics, requires a skilled narrator to manage rules, timing and atmosphere. It often becomes difficult to find someone among the players that can or want to do that. Without this role, the experience can quickly lose its balance and engagement, especially for new or young players.

This project addresses this challenge by introducing an AI-powered narrator that transforms the game into an accessible, engaging and always available experience.
The solution is provided as a mobile application, so that each player can use only his smartphone. At the core of the app there is an **LLM** (*Large Language Model*) **narrator** designed to guide players through the game. Acting as an impartial storyteller, the narrator assigns roles, manages the phases and help building a new story every time. The app allows each player to focus only on strategy, social interactions and immersion. Different settings are introduced, in order to provide even more possibilities to customize each game session, adding also an easier and shorter version of the game for children.

From a UX point of view, the interface is very simple and clear, in order to focus players' attention on the social interactions in the physical shared space, more than on the screen.

The final goal of this project is to make Lupus in Fabula more accessible, engaging and enjoyable. The app aims to preserve the social essence of the game, while enhancing is playability, providing a new version of the game, that relies on AI implementation, without giving up on social interactions. It offers a modern reinterpretation of the classical game that enables players discover the potential of technology as an ally more than a substitute.

From this starting point, the project could be developed more in the future, adding new settings and options to customize the experience and narration.

2 Requirements

This chapter outlines the requirements that guided the design and development of the application. The users' needs are the starting point to envision the possible solution and develop the system. In the next paragraphs stakeholders, user needs, context, constraints and goals are fully described and analyzed.

2.1 UNG Diagram

The *UNG Diagram* (User Needs Goals Diagram) helps organizing and correlating all the elements previously mentioned, that are useful for the requirements analysis: stakeholders definition, users' needs, goals of the project, constraints and general context.
Each cathegory will be addressed in the next paragraphs.

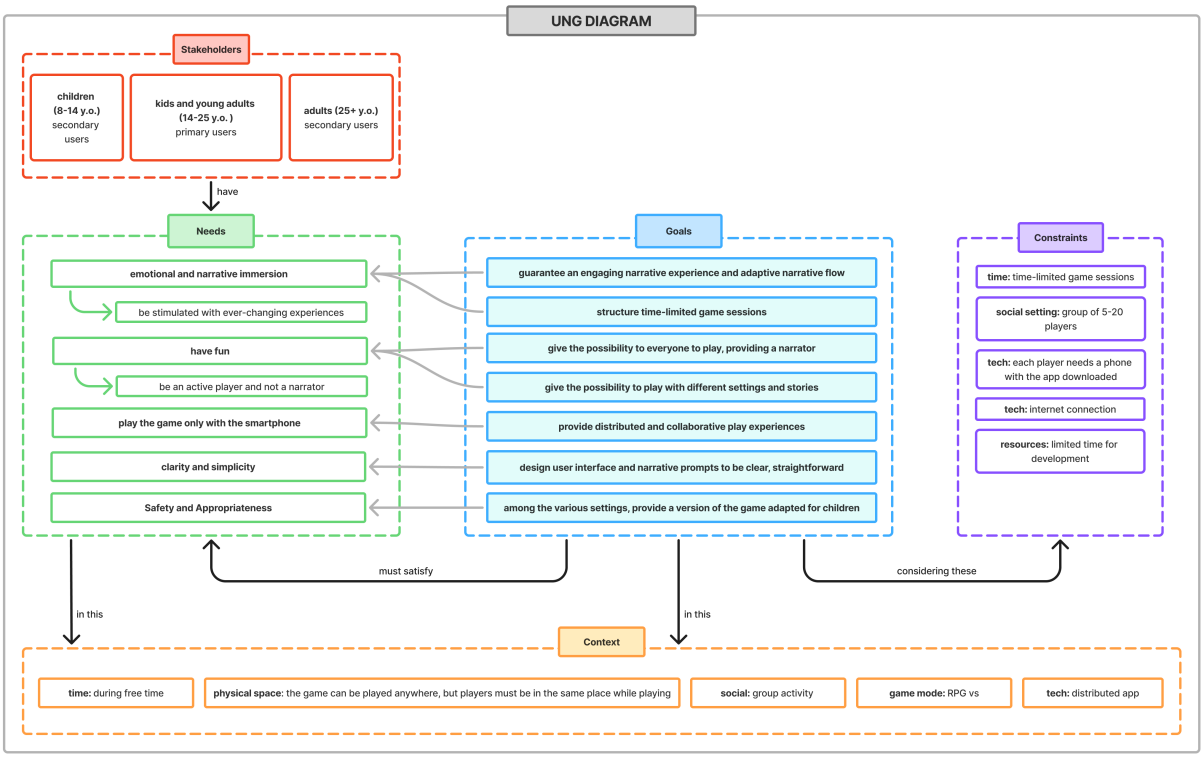


Figure 2.1: UNG diagram

2.2 Stakeholders

This project addresses teenagers and young adults (14-25 y.o.) as the main users, as they are usually more interested in playing this kind of games.
However, it considers also kids (8-14 y.o.) and adults (25+ y.o., especially parents or educators) as secondary users, as the game could actually be played by almost everyone. These three categories together form the set of stakeholders.

Based on these categories, three users archetypes were defined using the *persona* methodology. Personas are a qualitative design instrument for identifying fictional, yet realistic, characters representing distinct user groups. By consolidating patterns, expectations, and behaviors, personas facilitate the interpretation of complex sets of users and guide the design choices keeping in mind the users' needs as the main starting point.

Personas definition: [1]

- LEO, 10 y.o. -secondary user
  Leo is an energetic pre-teen who loves mysteries and games like Among Us and Roblox. He often plays with his cousins during family gatherings and loves being part of a story. He finds it hard when adults have to explain complex rules, so an AI narrator that takes charge feels magical to him.

- MATTEO, 23 y.o. -primary user
  Matteo is a busy student which loves party games like Mafia, Werewolf, and Jackbox, but often ends up being the narrator instead of playing. When he heard about an AI narrator that could take over that role, he got instantly excited: finally, everyone could participate equally.

- MARTA, 38 y.o. -secondary user
  Marta used to love playing Mafia with friends in university, but since having two kids, time is scarce. She's looking for games the family can play together that don't require setup or long instructions. An AI narrator that guides and entertains makes family game night much easier.

2.3 Needs of the Stakeholders

The users' needs are related to all stakeholders, even if some of them are more specific for some categories. Primary needs are listed below with some sub-needs where needed to clarify and better explain the challenge.

Users needs:

- **EMOTIONAL AND NARRATIVE IMMERSION**: Users seek an engaging and evolving narrative experience capable of sustaining attention over time. Emotional involvement enhances participation and fosters a deeper connection with the game dynamics.
  - SUB-NEED: Be stimulated with ever-changing experiences

- **FUN**: The experience is expected to be entertaining and engaging, encouraging active participation rather than passive listening. Enjoyment is a key factor in maintaining motivation and repeated use.

[1] Personas' descriptions have been generated with the help of MULCA AI tool

  - SUB-NEED: Be an active player and not a narrator: Sometimes being the narrator can be exciting, but when someone is not able or does not want to have this role, it can become something that slows down the game rhythm and discourage players to do multiple sessions.

- **POSSIBILITY TO PLAY THE GAME ONLY WITH THE SMARTPHONE**: Users require the game to be fully playable using a single device, without the need for additional tools or materials. This ensures accessibility, portability, and ease of use in different contexts.

- **CLARITY AND SIMPLICITY**: Clear rules and intuitive interactions are essential to ensure immediate understanding of the gameplay. Simplicity reduces cognitive load and allows users to focus on the experience rather than on instructions. (Especially for young players)

- **SAFETY AND APPROPRIATENESS**: The content and interactions must be suitable for users of different ages and backgrounds. Ensuring safety and appropriateness contributes to a comfortable and inclusive gaming environment. (Especially for young players)

2.4 Context

The project refers to the context of RPG (Role Play Games), as it is a revisitation of one of the most popular of those. It concerns people's free time as it is developed to be a way of entertainment, a fun activity.

One of the core elements of this project is to maintain the social interactions that are characteristic of the original Lupus in Fabula game, so it is important that the app is used by people that are together in the same physical space while playing, but it can be used anywhere at anytime, since players only need their phones and no others supplies.

The LLM narrator gives the possibility to each player to play, reducing the total minimun number of players compared to the original game, in which one of the players must take the role of the narrator (meaning that one more person is required to reach the minimun number). Anyway, to make the game session more engaging, this version is designed for at least 5 players, also to maintain the social nature of this game.

Having the personas in mind, three possible scenarios can be described to better understand the context of this project: [2]

- SCENARIO 1 (LEO, 10 y.o. -secondary user)
  Leo was buzzing with excitement as the weekend arrived, bringing the promise of a joyous family gathering at his grandparents' house. As soon as he stepped in, and after exchanging all the customary greetings and hugs, he spotted his cousins gathered around a table, already chatting animatedly about what games they might

[2]Scenarios descriptions have been generated with the help of MULCA AI tool

play. "Can we play Lupus in Fabula?" Leo eagerly suggested, recalling the app his parents had recently installed on his tablet. After promptly asking for and receiving permission from his mom to play for a little while, Leo energetically led his cousins to a cozy corner of the living room. He grabbed his smartphone and opened the app, effortlessly navigating to the "Play with Friends" option. With a few swift taps, he set up the session, entering the number of players and crafting a playful password. After typing in his nickname, he watched as each of his cousins joined the lobby, eagerly punching in their own nicknames and the secret password, all giggling with excitement. Once everyone was settled, Leo selected the child-friendly version of Lupus in Fabula, choosing bright and adventurous graphics that promised fun and playful storytelling. The screen shimmered to life as the charismatic AI narrator took center stage, greeting them with an engaging introduction to the game. Voices filled the room with laughter and exclamation as they each learned their roles through animated prompts, perfectly designed to keep them self-sufficient and in the spirit of playing. As the game progressed, Leo found himself deeply immersed, strategizing and bluffing with flair. With each round succinctly paced, Leo and his cousins could savor multiple narratives, each wrapping them in fresh twists and vivid plots. As the session wound down, the AI narrator shared playful feedback and awarded fictional storytelling badges, making Leo beam with pride and determination to outsmart his cousins the next time. Finally, the AI concluded with an eloquent wrap-up, summarizing highlights and prompting a chorus of delighted chatter among the young players. Leo, his screen time nearly up as per his parents' reminder, closed the app with a satisfied sigh. The game had not only provided an exhilarating adventure but also left a lingering excitement about stories and strategies for the games to come.

- SCENARIO 2 (MATTEO, 23 y.o. -primary user)
  Matteo glanced at his phone, a brief lull in his part-time shift giving him just enough breathing room to hatch a plan. He craved the kind of spontaneous, fun-filled evening that only a good game night could deliver. Within a few taps, his phone became the catalyst for camaraderie, as he messaged his closest friends, coordinating schedules and effortlessly carving out a time post-shift that everyone could join. With time set and excitement brewing, Matteo opened the Lupus in Fabula app. Its intuitive interface made quick work of setup, primed for the impromptu fun to come. Matteo appreciated the efficient design—it meant more time for thrills and less for logistics. As friends began to join in via their smartphones, Matteo felt a familiar, exhilarating anticipation. Invites flew out, and within moments, the digital lobby filled with friends eager to dive into the loop of mystery and mirth. The AI narrator swiftly took command, weaving an introduction that blended clarity with charm. Matteo watched his friends' faces light up, each reassured and entertained by the effortless explanation of game roles and rules. For the first time in ages, Matteo found himself just another player, free from the duties of narrator. The game unfurled with lively twists orchestrated by the AI, every moment a fresh challenge. Matteo reveled in the strategic banter and playful bluffing, his keen mind delighting in the game's agility

and depth. Laughter and mock-serious debates filled the room, energizing the group with shared purpose and enjoyment. As the final round concluded and the AI artfully narrated the game's end, Matteo couldn't help but smile with satisfaction. The wrap-up sparkled, highlighting humorous blunders and brilliant strategies, sparking post-game banter that lingered as sweet echoes of a night well-lived. Later, group chats buzzed with memes, recounting the epic plays and comedic moments that etched the evening into their shared history. Social media lit up with Matteo's captures of the night, the debriefing already a treasured ritual to reinforce the bonds of friendship and the love of the game. With a feeling of fulfillment, Matteo tucked his phone away, reflecting on how the evening's success bodes well for many more game nights to come, enhanced by AI's magic and the simple joy of being present with friends.

- SCENARIO 3 (MARTA, 38 y.o. -secondary user)
  Saturday morning was bustling in Marta's household as sunlight spilled through the windows of their cozy Naples apartment. With a long weekend stretching ahead, Marta decided it was the perfect opportunity to plan a family game night. Over breakfast, she animatedly described the idea to her husband and gathered their two children, ages 8 and 10, and her sister making sure everyone was on board. The mere mention of the new AI-driven game, Lupus in Fabula, sparked immediate excitement in the kids' eyes. The anticipation built steadily through the day, and by evening, the living room was a canvas primed for fun. Marta deftly launched the app on her smartphone, selecting "Play with Friends" and opting for the "Child-friendly" mode—purely designed to delight and engage her little ones. A quick few taps set up the game session; she created a lighthearted code and crafted a whimsical nickname. Her family eagerly joined, inputting their own nicknames and smiling as they were welcomed into the lively digital lobby. The game began with the AI narrator seamlessly introducing the rules and objectives in a voice imbued with warmth and humor, effortlessly captivating the children. Marta relished in being an active participant, freed from the constraints of moderation, engaging wholeheartedly in the narrative. As they took turns, the room filled with giggles and spirited storytelling, each family member playing their part in the shared adventure. As the game unfolded, alliances were formed, and playful strategies hatched, reinforcing bonds through laughter and teamwork. Marta's heart swelled watching her children immerse themselves, glowingly taking the lead in colorful, imaginative sequences. When the last chapter of their game came to an end, the AI brought them together with a delightful summary, highlighting their antics and the laughter shared. The family collectively reflected on the experience, animatedly recounting blunders and bold moves, pondering over what strategies could be tried in the future. As the night winded down, Marta and her husband savored the success of the evening. They marveled at the joy on their children's faces and decided there would certainly be another game session soon. After tucking the children into bed, both satisfied and brimming with ideas, Marta realized these would be the moments her family would cherish and carry forward—laughter, learning, and love, all wrapped in the adventure of a simple, wonderful game experience.

The analysis of these scenarios helped also as a starting point for the definition of users' tasks and workflows, explained in chapter 4.

2.5 Constraints

Constraints that have to be taken into account for the development of the project are listed below divided into categories.

Technological constraints:

- SMARTPHONE: each player needs a phone with the app downloaded
- INTERNET CONNECTION: internet connection is necessary for each player to connect with others and join the lobby

Social constraints:

- GROUP SETTING: to play the game the expected and necessary minimum number of players is 5. The app is developed to host a maximum number of 20 players in the lobby.

Resources constraints:

- LIMITED RESOURCES: the project has to be developed in a limited time span, without a specific budget designated for that.

Time constraints:

- LIMITED TIME: each game session has to conclude within a limited time window in order to match limited attention span and engagment rate of players.

2.6 Goals

Considering all the elements previously mentioned, especially having in mind users' need and expectations, the project's goals have been defined and are listed below.

2.6.1 Stakeholders'

Starting from the stakeholders' needs and expectations, the project goals have been defined and are listed in this paragraph:

- GUARANTEE AN ENGAGING NARRATIVE EXPERIENCE AND ADAPTIVE NARRATIVE FLOW
- STRUCTURE TIME-LIMITED GAME SESSIONS
- GIVE THE POSSIBILITY TO EVERYONE TO PLAY ACTIVELY, PROVIDING A NARRATOR

- GIVE THE POSSIBILITY TO PLAY WITH DIFFERENT SETTINGS AND STORIES
- PROVIDE DISTRIBUTED AND COLLABORATIVE PLAY EXPERIENCES
- DESIGN USER INTERFACE AND NARRATIVE PROMPTS TO BE CLEAR, STRAIGHTFORWARD
- AMONG THE VARIOUS SETTINGS, PROVIDE A VERSION OF THE GAME ADAPTED FOR CHILDREN

2.6.2 Technological

- **GUARANTEE REAL-TIME SYNCHRONIZATION OF GAME STATES ACROSS ALL CLIENTS**: The infrastructure must ensure that all connected devices transition between game phases—such as story generation, player input, and voting on narrative branches—in perfect unison. This synchronization is critical to maintaining a coherent collaborative experience and preventing state discrepancies among the group.
- **MINIMIZE LATENCY IN GENERATIVE AI RESPONSES AND NARRATION**: The architecture must optimize the communication pipeline with the Large Language Model to ensure that narrative descriptions and game events are generated and delivered within seconds. This is critical to maintaining the game's rhythm and preventing drops in player engagement due to long pauses.
- **ENSURE SCALABILITY FOR MULTIPLE CONCURRENT GAME LOBBIES**: The backend infrastructure must support simultaneous execution of numerous distinct game sessions without performance degradation or data cross-contamination. This ensures that an increase in active users does not impact the responsiveness or stability of individual private games.
- **IMPLEMENT ROBUST CONTENT FILTERING AND SAFETY GUARDRAILS**: The system must technically enforce strict constraints on the LLM's output to prevent the generation of inappropriate, violent, or unsafe content. This capability is essential to technically support the "Child-friendly" mode and ensure a safe environment for all stakeholder demographics.
- **PROVIDE A CONSISTENT CROSS-PLATFORM EXPERIENCE**: The application must function identically across different mobile operating systems to allow mixed groups of users to play together seamlessly. The technology must abstract device-specific differences to ensure the UI and game logic behave predictively on any supported smartphone.
- **ENSURE CONNECTION STABILITY AND GRACEFUL RECOVERY**: The system must be resilient to temporary network fluctuations common in mobile

environments. It should allow players to automatically reconnect to their active lobby and retrieve the current game state without disrupting the session for the rest of the group.

3 State of the art

This chapter presents the current state of the art regarding the use of **Large Language Models** as **narrators**, as well as the application of the *Lupus in Fabula* game in research contexts.

3.1 Games and LLMs: A short introduction

Games, including both board games and videogames, serve both as a source of important benchmarks for research in AI and as an important application area for AI techniques. Almost every game uses some kind of AI technology, researchers and developers are trying to figure out the best make use of recent advances in AI [1].
*Large Language Model*s are typically presented as conversational agents, which often invites the public to give them anthropomorphic qualities, such as reasoning and creativity. An LLM can operate within the game as player, non-player character and as a *Game Master* (*GM*) controlling the flow of the game.
Very often, *role-playing game* are chosen to evaluate the performance of LLMs in this field. RPGs are games in which players assume and play out the roles of characters in a fictional setting. Players take responsibility for acting out these roles within a narrative, either through literal acting or through a process of structured decision-making regarding character development.
A subcategory of these games are the so-called *tabletop role-playing games* (as known as *TTRPGs*), a type of role-playing game that relies solely on narration to describe the characters' actions or situations through dialogue, unlike games such as Baldur's Gate 3, where the story is also told through cutscenes and in-game animations. These games are usually divided into campaigns, which last several sessions, each of which can last several hours and is played in person around a table. The most famous example of a TTRPG is *Dungeons & Dragons* (*Dnd*).

3.1.1 The role of the Game Master

In the world of RPGs, the Game Master plays a fundamental role, as they are responsible for running the game and devising the story.
GMs are often, at least partially, in charge of game design in terms of world-building and designing challenges [2]. The role of a GM can be a taxing endeavor, with them being given the task of world-building, game design, and information tracking.
Given the demanding task of GMs and the generative capacity of LLMs, research immediately focused on AI-driven tools to facilitate their task. For example, Tang et al. [3] interviewed GMs to identify the requirements for computational support and found that the most important features were inspiration, controllability, and ease of use.
An issue identified in the selected papers, is the amount of dense narrative information GMs must sort through to run TTRPG sessions. To assist with this issue, interactive flowcharts of narrative events, information highlighting, and summarization tools were created.

For example, CALYPSO is an AI co-GM tool running on a Discord server created by Zhu et al.[4] to support GMs, relieving them of certain tasks, such as generation of random encounters or brainstorming ideas, so they can focus on more cognitive demanding issues, such as improvising NPCs (non-player characters) dialogue.

3.2 LLMs as narrators

The behaviour and capabilities of LLMs in the role of narrators in RPG games have been studied for years. One of the first noteworthy examples is AI Dungeon[5], a text adventure that uses a fine-tuned version of GPT-2. The story is continued by the LLM thanks to the user's text input, imitating the style of a human GM. Hallucinations are inevitable given how the world is described to the machine; LLMs lack grounding, so the text they generate is detached from constraints of reality, but this is not always negative: hallucinations can better describe characteristics of creatures, although in some cases it has added wings to canonically unwinged creatures.
A similar experiment was done by Aoki et al. [6], they used a simple dungeon setting with 3 rooms: a spawn point for adventurers, a room with a dragon and a treasure room, which was the goal. Adventurers where generated by LLM (GPT-2 same as AI Dungeon) and their goal is to reach the treasure room, they are not obliged to kill the dragon, the LLM narrator has to write a story based on this setting. Researcher run the simulation several times and noticed that stories had logical conflicts, repetition and lack of coherence, furthermore these stories were biased through peaceful endings, such as adventurers and the dragon that reached an agreement, consequently there were few descriptions of battles. Their work proposed to address this issue by training models on combat descriptions or on tragedies genre works.
One of the most important aspects in games is immersion, as a cognitive response in players. This type of response is often sub-divided either as a response to perception, challenge or narrative, which is a key component in feeling immersed [7]. Immersion in RPGs and tabletop games is also linked to the environment in which the game is played, for example, if my fellow adventurers in a DnD campaign are not participatory, my immersion will suffer even if the story is very well written.
One of the considerations made by Maisto [8] in his analysis is that the formal register used by LLMs in their responses is not suited for RPG sessions, characterized by an informal oral register. Olson[9], for example, affirmed that spoken discourse is context-bound, while written discourse is decontextualized. Cohesion, which is really important when writing sounding stories, is lexically established in writing but relies on paralinguistic and non-verbal channels in speech.
Another challenge is that currently LLMs are trained to be highly compliant to the users' requests, because human Game Masters frequently curb the more exotic player requests, which could drastically diverge from the game narrative, or which would result in an unrecoverable disruption of a required sequence of game events. An LLM Game Master would try to accommodate for even the most bizarre requests, with little consideration for the consequential impacts to any predetermined game events.

Another issue to take into account is that it is not yet possible to apply LLM narrators to real-time applications due to the long response time of the models, which can sometimes exceed 30 seconds.

3.2.1 Interactive digital narratives

Interactive digital narratives (IDNs) are stories where users influence outcomes through their choices and actions. This type of narrative is fundamental in games such as DnD but also, albeit to a lesser extent, in Lupus in Fabula, where the follow up of the narrative is chosen based on the actions performed during night, or day, phase. In IDNs , LLMs create a new dynamic where authors guide intelligent agents, while narratives emerge from player interactions. LLMs have the potential to deviate from the starting point of the story thanks to user inputs, but their creativity is not fully utilized. Often works lack richness (and feel boring or monotonous) because LLMs have a homogenized lexical style. Orchid [10] is a framework proposed by Wu et al. that addresses two main hypothesis:

1. LLM-based IDNs created with Orchid facilitate an open-ended collaboration between authors, machine creativity, and player interactivity to generate emergent textual content;

2. Orchid supports the creation of richer narrative structures beyond traditional linear formats commonly found in many LLM-related experiences, such as in AI Dungeon.

3.2.2 Nova

Nova is a framework developed by Cauàs et al. [11] which addresses the problem of coherence and impact of narratives created by artificial authors. These narratives can describe events, provide commentary, and immerse the player in the game world. Another significant obstacle is identifying the relevant points to narrate, avoiding an overload of information that could dilute the narrative experience. Coherence in this context means that the narrative generated by the LLM makes logical sense within the game's framework and maintains continuity. Using Retrieval-Augmented Generation (RAG) on a diverse set of narrative examples, LLMs can generate stories that are contextually appropriate and engaging, but this limits the generation of narratives to the scope of examples provided during training, lacking creativity and originality. LLM-generated narrations also may lack variability, resulting in repetitive and monotonous storytelling that can quickly become stale for players. Nova integrates three key modules to assess these identified issues. The first one is the **Anomaly detection module** which serves as a trigger for beginning the narrative at specific salient points. The **RAG** module then generates the story, maintaining consistency during gameplay. The last module is the **Emotional** module which allows the LLM to craft more creative and engaging narration, making the storytelling immersive.

3.2.3 LLM narrators in Dungeon&Dragons

Dungeons&Dragons is one of the most important and influential TTRPG ever created, and in literature is used as an environment to evaluate LLM GM frameworks. The game master in DnD is called *Dungeon Master* (*DM* in short), he functions both as a moderator and a storyteller for the adventures that the players will role-play. Furthermore, DnD sessions are one of the most striking examples of IDN.
A first example of LLM narrator applied to Dnd is **Fable Reborn**, an interactive storytelling environment that employs a *virtual DM* (*VDM*) in the production of fictional stories by adapting rules from the DnD 5th edition (5e) playbook, proposed by Ang et al. [12]. This framework obtained mixed results, underlining a difficulty in generating stories from the fantasy genre with unrealistic events, although the story was coherent, hallucinations and errors in characters names or places limited the immersion. Furthermore the presence of high computational modules caused delays in the VDM's responses which can disrupts immersion, so process speed is crucial to limiting delays.
Tryason [13] evaluated the ability of ChatGPT in transforming storytelling into an interactive drama using dynamic and immersive narratives. He used 4 metrics to assess this ability:

- **Narrative Coherence**: ChatGPT was successful in producing narratives that were logically consistent and exhibited a clear narrative structure;
- **Narrative Immersion**: These measurements requires further testing, because he used a human facilitator which may have had an impact on the real satisfaction of the users. Nevertheless, narratives may have lacked the depth and immersive qualities necessary to fully engage the players and transport them into the game world;
- **Player Engagement**: ChatGPT was successful in maintaining a reasonable level of engagement, capturing the interest of the players and keeping them actively involved in the game;
- **Adaptability**: ChatGPT demonstrated the ability to adapt to various player actions and make appropriate adjustments to the narrative, enhancing the overall play experience.

Patel et al. [14] saw that most AI story generators operate in isolation without incorporating summarization for context retention. This means that, sooner or later, the model's context window will run out and it will inevitably lose data, including important one. That's why in their model *NAT1* they introduced the **Summarization Module** which, every n turns, compiles a concise summary of the session's key events. This summary would be used as the history for story generation model, reducing tokens used for context, and in turn, lengthening the session.

3.3 Lupus in Fabula evaluation framework

Lupus in Fabula, also know as *Mafia Game* or *Werewolf*, is a classic social deduction game where each player is assigned an hidden role belonging to two opposite factions: *villagers* or *werewolves* [15]. The villagers aim to eliminate all werewolves through public voting, while the werewolves group strives to reduce the number of villagers to zero, by eliminating them over night. To add complexity to the game, several roles can be added to both factions, such as the seer, belonging to the *villagers* having the ability to check a player's affiliation over night. The game is composed of two major phases: the night phase, where werewolves and special roles play, and the day phase, where all player discuss on who might be a werewolf and choose one player to eliminate through lynching. These phases are coordinated by a narrator, who is responsible for determining the order of the various roles, chairing the discussion and formalizing the result of the two phases and the entire game.
In recent years *Lupus in Fabula* has been used as a framework to evaluate LLM models in an environment full of false information, where nothing can be trusted.

3.3.1 Agent-Based framework in Unity

Poglitsch et al. [16] built a goal-based evaluation framework in Unity for LLMs playing Lupus in Fabula. Each model is assigned a goal based on the role obtained during the game. The objectives are:

- **Werewolves**: Eliminate villagers until their numbers equal or exceed the non-werewolf players;
- **Villagers**: Identify and eliminate all werewolves.

The implemented phases are:

- **Night Phase**:
  - Werewolves choose a player to eliminate;
  - The seer learns the role of one player.
- **Day Phase**:
  - Players discuss and vote to eliminate a suspected werewolf.

To induce the LLM to plan its actions they asked it to state a goal and then generate 5 steps to reach that goal, this operation is done every day at dawn. The planning ability worked as expected with the models that fulfill the criterion of making sounding plans. Unfortunately those plans sometimes included already eliminated players. The study assesses how well these models handle the intricate state systems typically seen in computer games, despite the unreliability of the agents' memory systems.
To reduce the impact of hallucinations that polluted the data gathered in the previous

framework Watanabe at al. [17] proposed to add a logical structure module separated from the text generating one. The results of subjective evaluations showed that the proposed method, which included a logical structure, outperformed the baseline that lacked such structure, enabling more appropriate reasoning.

3.3.2 Werewolf Arena

Werewolf Arena [18] is one of the most cited LLM evaluation framework which is based on Lupus in Fabula. According to the authors "*The inherent information asymmetry of Werewolf, where only some players possess incomplete knowledge of others' roles, mirrors dynamics of real-world social interactions. Our Monte Carlo simulation highlights the critical role of strategic communication in overcoming this asymmetry. Without it, Villagers win a mere 1.2% of 100,000 simulated games.*" To assess difficulties in managing communication, as a random speaking order would have implied, they introduced a dynamic turn-taking system where players bid to speak. To balance the evaluation each model should play both *Villager* and *Werewolf* roles.
The resulted communication style of different models is:

- **GPT-4 players**: Favored longer, more formal utterances, often emphasizing collaboration and consensus-building. They exhibited a relatively narrow emotional range in their dialogue, rarely expressing strong emotions.;
- **Gemini 1.5 Pro players**: Communicated with shorter, less frequent utterances characterized by greater emotional expression. They frequently incorporated humor, sarcasm, and expressions of frustration or suspicion into their dialogue.

4 Solution - UX Design

This chapter describes the solution proposed and the design process for developing the User Experience and User Interface.

4.1 General approach

The proposed solution is a mobile application that helps people play Lupus in Fabula in a more engaging way, providing a virtual narrator.
In the next pages the design process and the motivations behind every choice will be addressed and explained in details. It is important to keep in mind that while working on the UX design, it was crucial to remember everything mentioned in chapter 2: the requirements served as a guide during the entire process.

CHOICE OF THE TECHNOLOGICAL SUPPORT

In order to make the game more accessible for everyone to play it everywhere, the solution was developed as a mobile application for smartphones. In our society, the smartphone is the tool that everyone has with them everyday, representing an easy way to play the game without additional supplies. It is accessible also by children who can easily ask their parents' permission to use their phones.

CHOICE OF THE DESIGN STYLE

One of the main goals of this project is to look at the technology as a support for already existing social dynamics. Starting from this assumption, the design of the user interface has been developed to be useful and intuitive, but not too rich. This was a choice made so that the screen will not be the main focus in the context of interaction with other players, that remains the core of playing together and having fun with others.

4.2 Interaction and workflow

To better understand the user interaction with the app, here is attached the Primary User Task Diagram. In the next pages it will be analyzed in details, associating every action to the screen interface.

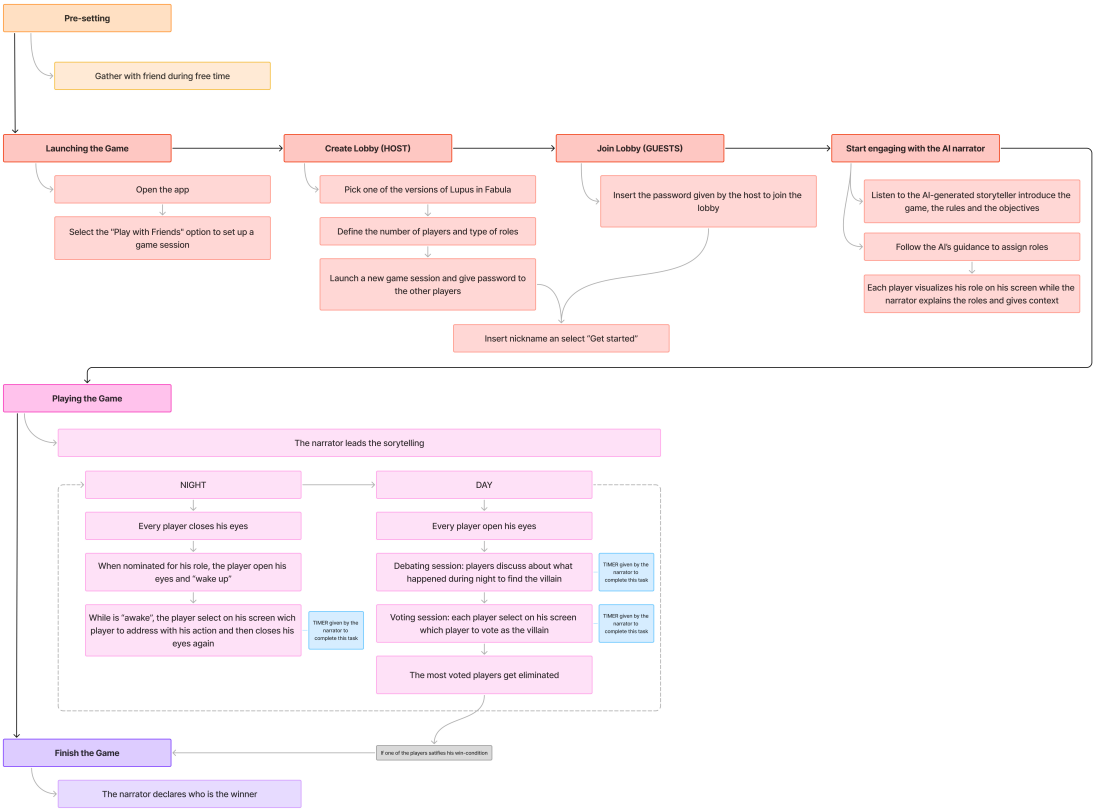


Figure 4.1: TASK diagram

4.2.1 Pre-Setting

For what concerns the Pre-setting phase, the solution provide a faster and easier alternative to the classical game: the only task that the user has to complete is to gather with friends in the same place with their smartphones. The use of the app eliminates the struggle to convince some of the players to play as narrator and the difficulty to remember all the roles and rules (if the players do not have the board game cards with them at the moment, which is very commen due to the spontaneous nature of Lupus In Fabula game sessions).

4.2.2 Game-Setting

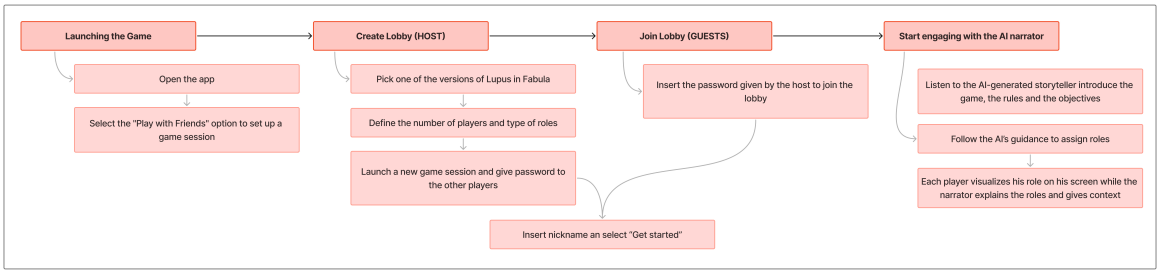


Figure 4.2: Game-Setting

- Opening the application, the user can simply tap on the screen to launch it and play with friends.
- The user can now choose to host the game, creating a new lobby, or join an existing one created by their friend. The choice is made by tapping on the corresponding button.
  - **HOST-CREATE LOBBY**
    The host has to: insert the number of players, select the setting of the game among various versions available (Classical, Vampire, Zombie, Children), create and share a password for the lobby, choose his nickname for the game. It is also possible for the host to customize the settings of the game by choosing which roles to add. Each role is explained in an additional page that can be open when needed, pressing the button "roles". Secondary roles can be added only one per type. [3]

[3] In children version, in order to make the game easier and shorter, the host can not choose what roles to add and the children will play the game only with primary roles: wolves and villagers, automatically counted based on the number of players.

Figure 4.3: Versions options (icons)

  - **GUEST-JOIN LOBBY**
    The guests have to join the lobby created by their (host) friend by inserting the shared password and choosing their nicknames.

- Only when the number of participants in the lobby reaches the number of players indicated previously by the host (meaning the everybody succeded in connecting to the lobby), it will be possible to select the button "start the game" and launch the session.

- Finally, the narrator presents himself, sets the mood to start the game and prepares players to recieve their role. When everyone is ready to play, they can tap on the screen and be able to discover their role in the game.

- The players can visualize their roles on their screens. They will find also an explanation of their powers, their goal and what actions they have to do in order to win the game.[4] This phase is regulated by a timer, automatically controlled by the app, to give everyone time to read and understand their position but, at the same time, maintain a proper and engaging timing.

[4]Roles are randomly assigned, so that everyone can have the chance to play different roles and interpret different characters in different game sessions. It is important that each player keeps their role secret in order to enjoy the mystery of the game, as in the original version of Lupus in Fabula

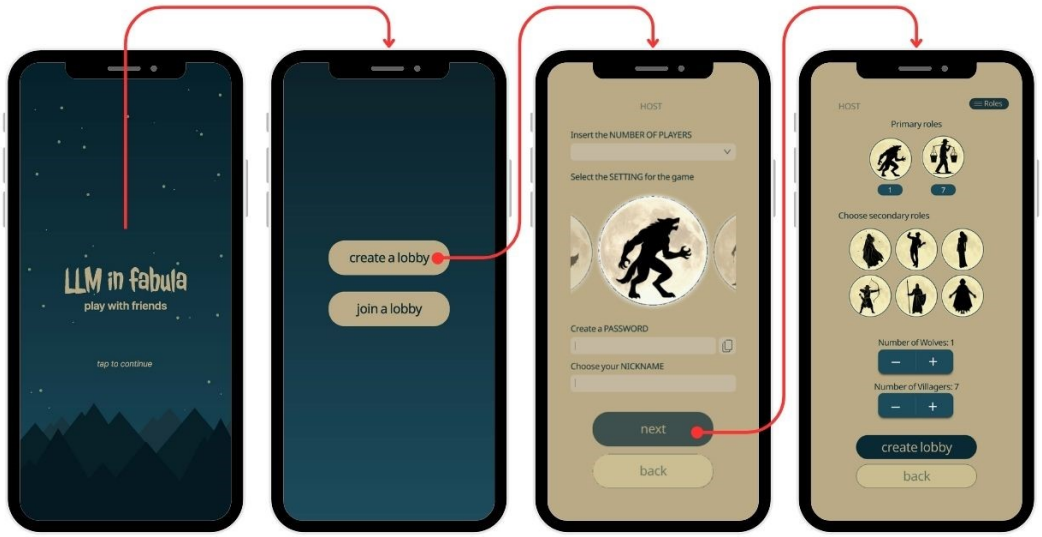


Figure 4.4: Game Setting workflow

4.2.3 Game Session

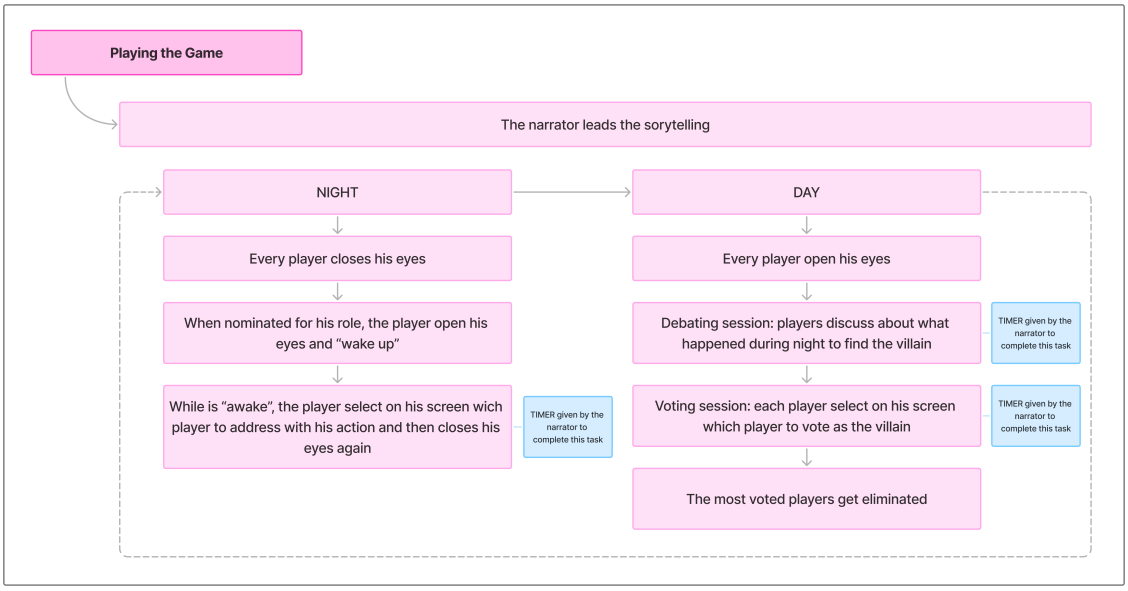


Figure 4.5: Game Session

Each game session is conducted by the LLM narrator that manages the phases and guides the players through the game. The narrator voice is audible from the host phone

and each player can visualize the script on their phone.
The game is characterized by the alternation of two main phases: DAY and NIGHT, as in the original game.

- **NIGHT**
  The night is the most crucial phase, during which every role can use his power and complete the action. As in the original version of Lupus in Fabula, the players are asked to keep their eyes closed during this phase, so that they can not see the other players completing their actions. The narrator will tell them when is their turn to open their eyes and complete their task by voting on their screen who they want to address with their action. Each player will visualize the voting interface only when it is their turn to complete the action and for a limited period of time to maintain the timing engaging.

- **DAY**
  When the day comes, the narrator will ask everyone to open their eyes again. This phase is when the social interaction really begin, since the players are asked to discuss to discover the villains and debate to prove their theories. The players are also asked to vote on their screens one player to eliminate. This phase is limited in time too. The narrator then reveals who got most votes and ended up eliminated.

4.2.4 End of the game

The game session ends when one player satisfies his win condition. In this case, the narrator conclude the story and announces the winner(s).

WIN CONDITIONS:

- **THE GOOD ROLES WIN**: if all the wolves/vampires/zombies get eliminated.
- **THE VILLAINS WIN**: if the number of good characters is less or equal than the number of wolves/vampires/zombies.
- **SPECIAL ROLE WIN**: the madman/thrall/contaminated wins if eliminated by other players during voting phase. [5]

4.3 User scenario

Starting from the scenarios described in chapter 2 , this section contains an example of a possible scenario of a secondary user (kid). This scenario focuses on the User Interaction and workflow with the application.

SCENARIO EXAMPLE (LEO, 10 y.o. -secondary user)

[5] This win condition does not apply to children version, that is limited to the first two conditions (GOODvsEVIL) and has no special roles.

1. 2. Leo opens *LLM in Fabula* app on his smartphone, ready to play with his cousins.

3. He chooses to create a new lobby (Host role).

4. Leo selects the child-friendly version. He defines the number of players, he creates and shares a simple password with his cousins. Leo inserts his nickname. He starts the lobby and waits for others to join.

5. Leo watches as each cousin: opens the app, selects "Join a lobby", inserts the password, chooses a nickname and enters the lobby he created. Leo confirms that all players are present, he taps "Start the game" and they are ready to play.

6. They all listen to the AI narrator's introduction.

7. The AI narrator assigns roles to each player privately. Leo is the wolf for this game session. He is excited to know that he plays a crucial role. He reads his role description silently and understands his objective.

8. Leo closes his eyes when instructed as everyone else when the night comes.

9. 10. He waits to be awakened by the narrator: he follows the instructions, makes a choice and closes his eyes again. He selected Sofia to be banned by the village.

11. 12. Leo opens his eyes with the group. He listens to the narrator's summary of the night and everyone discover that Sofia disappeared.

13. Now it's time to discuss and vote who is the wolf. Leo discusses what might have happened with his cousins. He bluffs or shares "suspicions" strategically.

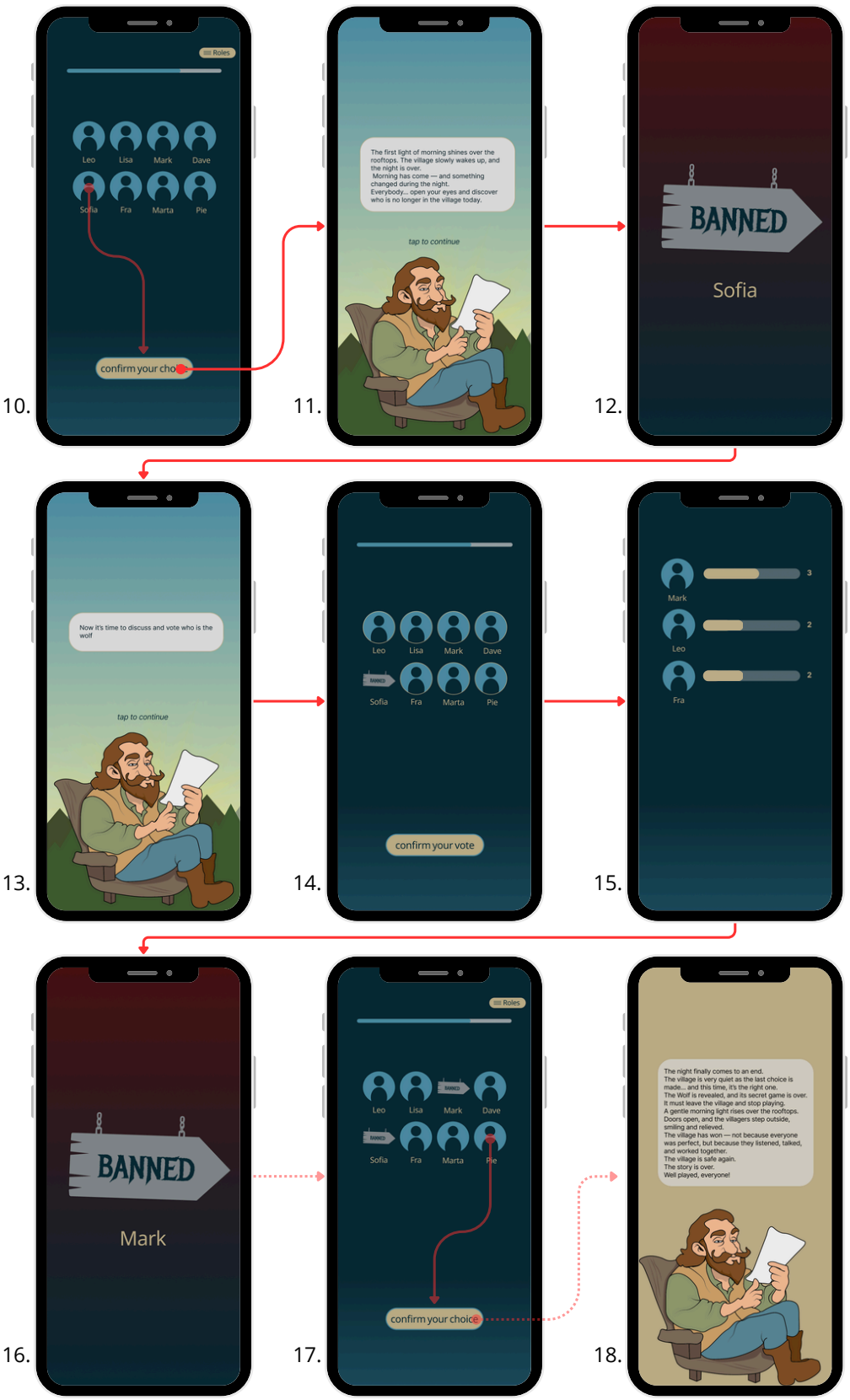
14. The voting page appears and Leo votes for Mark, since somebody else accused him.

15. 16. The results of the poll are showed on the screen and Mark gets banned from the village, since he is the most suspected player.

17. The game session goes on and the narrator tells everyone to close their eyes again since the night has come. Leo opens his eyes when called by the narrator to play his action. He decides to select Pie because he was the one that accused him in the first place during the discussion phase and he wants to get rid of his enemy. He selects the player on the screen, confirms, and goes back to sleep, closing his eyes.

18. The day comes and the narrator announces that Pie disappeared. Another time, the discussion takes place and now everyone seems to be sure that Leo is the wolf. His choice to eliminate his first enemy has shown his real intentions to other players, that vote him to be banned from the village. The results are showed and Leo is eliminated. The narrator concludes the story by announcing the victory of the village!

1.
LLM in fabula
2.
LLM in fabula
play with friends
tap to continue
3.
create a lobby
join a lobby
4.
HOST
Insert the NUMBER OF PLAYERS
Select the SETTING for the game
Create a PASSWORD
Choose your NICKNAME
next
back
5.
Roles
Leo
Lisa
Mark
Dave
Sofia
Fra
Marta
Pie
start the game
6.
Hello, everyone! I'm your narrator — the voice that guides the story and helps the game move along. Welcome!
This is a small, friendly village, full of people who work, play, and laugh together every day. Everyone seems happy... or at least they seem to be!
But when night arrives, the village becomes very quiet. Secrets start to appear, and some players are hiding something special. Are they just villagers... or something else?
Get ready!
It's time to discover your role and start the adventure!
tap to continue
7.
WOLF
By day, you walk among the villagers with a friendly smile, pretending to be just like everyone else. But when night falls, your secret role comes into play.
The Wolf acts quietly, choosing one player to mysteriously disappear from the village for a while.
During the day, you confuse the others, tell clever stories, and try to make them suspect someone else.
Your goal is to slowly take control of the village, until the wolves are the only ones left in the game.
8.
Night falls over the village. The sky turns dark and quiet, and the lights slowly go out.
Villagers... it's time to rest. Everyone, close your eyes.
The night has begun...
and while the village sleeps, the Wolves quietly wake up, ready to play their secret part.
tap to continue
9.
Roles
Leo
Lisa
Mark
Dave
Sofia
Fra
Marta
Pie
confirm your choice


10.
Roles
Leo
Lisa
Mark
Dave
Sofia
Fra
Marta
Pie
confirm your choice
11.
The first light of morning shines over the rooftops. The village slowly wakes up, and the night is over.
Morning has come — and something changed during the night.
Everybody... open your eyes and discover who is no longer in the village today.
tap to continue
12.
BANNED
Sofia
13.
Now it's time to discuss and vote who is the wolf
tap to continue
14.
Leo
Lisa
Mark
Dave
BANNED
Sofia
Fra
Marta
Pie
confirm your vote
15.
Mark
3
Leo
2
Fra
2
16.
BANNED
Mark
17.
Roles
Leo
Lisa
Mark
Dave
BANNED
Sofia
Fra
Marta
Pie
confirm your choice
18.
The night finally comes to an end.
The village is very quiet as the last choice is made... and this time, it's the right one.
The Wolf is revealed, and its secret game is over.
It must leave the village and stop playing.
A gentle morning light rises over the rooftops.
Doors open, and the villagers step outside, smiling and relieved.
The village has won — not because everyone was perfect, but because they listened, talked, and worked together.
The village is safe again.
The story is over.
Well played, everyone!

4.4 Conversational interaction

The conversational agent plays the role of the narrator of the Lupus in Fabula game, users do not interact directly with it but they can shape the direction of the story through their actions, in fact the narrator has the role to share global information, such as the death of a player, to everyone in the lobby.
Each setting in the game has an associated *initial prompt* which is passed every time the agent is called, to remind him the game's setting story, environment and thematic changes. Examples of this initial prompts can be found in the annex
There are two major ways in which we need to control the agent's output behavior:

- **Length limit**: to avoid excessive time loss due to agent's verbosity, the last part of the prompt is devoted to eventual constraints on the length of the response (e.g. [...] *Write a short paragraph of* ***no more than 40 words***[...]);
- **Family-friendly control**: the app includes a setting for children, where no violence, or inappropriate content should be shown, the initial prompt manages to filter this inappropriate content by giving alternatives to the agent, for example the death of a player becomes a ban to a vacation locality (e.g. [...] *Wolves and villagers* ***do not hurt anyone****-they only play tricks, make funny noises, or hide clues*[...]).

5 Implementation

5.1 Overview

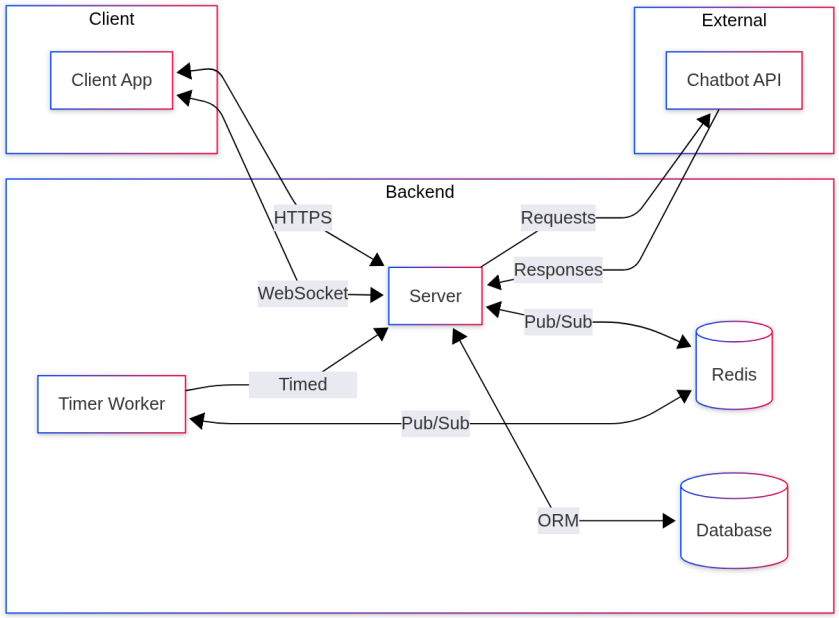


Figure 5.1: Architectural Design

The system is composed of 3 main modules:

- **Frontend**: contains an app developed in Flutter in order to take advantage of cross-platform functionality and make the app compatible with all devices;
- **Backend**: composed of the **server** and its modules:
  - **Redis**: used to store socket connections;
  - **Timer worker**: module used to control the timers;
  - **Database**
- **External**: includes all the APIs needed to support the system functionalities

The application and the server use both HTTPS and websocket (WS) to communicate, in this way the server stores only the connection of in-game players.

5.2 Front-end

The mobile application is developed in Flutter (Dart) to leverage its cross-platform capabilities. The user navigates freely only between the initial setup screens, where they can choose to join or create a lobby.

From that point onward, the entire application flow is strictly controlled by the server through a state-driven routing pattern. Each flow and subflow of the application is linked to a specific state of the server; when the server transitions states, it sends a message to the client containing the new phase. The app then automatically updates the screen to reflect this new server state, ensuring perfect synchronization between all clients.

5.2.1 User Flow and Main Features

The application is structured around a finite number of high-level navigation flows. These are managed strictly by the application state, ensuring that the user's view is always synchronized with the logical stage of the session.

Primary Screens and Flows

Based on the AppRouter configuration, the application manages 5 primary navigation flows, defined by the AppFlow enum.

- **AppFlow.title → PlayScreen**: The initial title screen where the application launches, serving as the entry point.
- **AppFlow.joinOrCreate → CreateJoinScreen**: The decision hub where the user chooses between initializing a new game session or joining an existing one.
- **AppFlow.create → CreateLobbyFlow**: A dedicated multi-step sub-flow. It guides the host through configuring a new game (setting themes, number of players) and manages the waiting room state while other players join.
- **AppFlow.join → JoinGameScreen**: The interface used by players to input a unique game code and connect to a match.
- **AppFlow.inGame → GameRouter**: A specialized, nested router that becomes active once the game officially begins. This router takes over responsibility for managing all specific game phases (e.g., Narrator Intro, Night Phase, Voting, Debates).

Accessibility via Text-to-Speech (TTS)

To enhance immersion and accessibility, the architecture includes a Text-to-Speech (TTS) module designed to automatically vocalize the AI Narrator's text during critical game phases. This feature ensures that the narrative is accessible to all players, including those with reading difficulties or younger children, and reinforces the atmosphere of a "narrated" story.

5.2.2 Architectural Implementation

To support the features described above, the frontend architecture relies on strict decoupling between business logic and presentation.

State Management

The core of the application's responsiveness lies in its State Management strategy. We utilize a centralized approach powered by the *provider* package, specifically relying on *ChangeNotifierProvider*.

- **The Single Source of Truth**: The entire application state is encapsulated within a singleton class named *GameState*. This class holds all critical runtime data, including the local *Player* object, the current *Game* instance, and the active navigation flow.

- **Injection Pattern**: As implemented in *main.dart*, the *ChangeNotifierProvider* wraps the root widget of the application (*MyApp*) directly within the *runApp* call. This pattern ensures that the single instance of *GameState* is injected at the very top of the widget tree, making it accessible to any descendant widget via the *context*.

- **Reactive UI Updates**: UI widgets are designed to be stateless regarding game logic. They subscribe to state changes using *context.watch<GameState>()*. When the *GameState* receives new data, it triggers *notifyListeners()*, prompting an automatic, targeted rebuild of only the relevant widgets.

Navigation

Navigation in "LLM in Fabula" departs from traditional imperative pushing/popping of routes in favor of a Centralized, Nested State-Driven Routing pattern.

- **High-Level Routing (*AppFlow*)**: The primary navigation logic is encapsulated within *AppRouter*. This widget listens to the *gameState.flow* variable, which is an enum of type *AppFlow*. It evaluates the current flow (e.g., *AppFlow.join*, *AppFlow.create*) and renders the corresponding high-level screen or sub-flow.

- **Nested Game Routing**: When *gameState.flow* transitions to *AppFlow.inGame*, the *AppRouter* hands control over to the *GameRouter*. The *GameRouter* operates as a sub-router. It listens to a different state variable - *gameState.phase* - to determine which specific game interface to display (e.g., *NightPhaseScreen*, *VotingScreen*).

- **Reasoning**: This state-driven approach avoids manual *Navigator.push/pop* calls, making the navigation robust and less prone to errors. It aligns the frontend directly with the Backend's Finite State Machine (FSM), preventing "navigation drift."

Backend Communication

To support the immersive, narrative-driven nature of the game, communication is handled exclusively via **WebSockets**, enabling real-time, bi-directional data flow.

The Intermediary Role: By design, *GameState* acts as the sole architectural intermediary for all WebSocket communication. It is responsible for:

1. **Lifecycle Management**: Establishing and maintaining the connection to the server.

2. **Event Interpretation**: Receiving raw JSON messages and interpreting specific game events (e.g., phase changes, chat messages, narrator prompts).

3. **State Mutation**: Updating its internal variables (*flow*, *phase*, *messages*) based on the interpreted event and subsequently invoking *notifyListeners()* to propagate changes to the UI.

Theming and UI Componentization

The UI architecture is built to be **Data-Driven** and **Extensible**, adhering to the DRY (Don't Repeat Yourself) principle.

Static Asset Management (*GameThemes*): The management of themes and icons is handled by a designed static utility class, *GameThemes*. Upon app startup (in *main()*), this class is responsible for asynchronously loading:

- **Configuration**: Theme settings and role definitions.
- **Mappings**: Icon definitions and asset paths.

Smart Componentization: The UI architecture specifies the use of reusable, "smart" components rather than static widgets. These widgets do not contain hard-coded display logic; instead, they receive abstract identifiers (like a role ID). Inside these components, the logic consults both *GameState* (for context) and *GameThemes* (for asset resolution) to dynamically render the correct visual asset - whether it be a standard Material icon or a custom image - at runtime.

5.3 Back-end

The system's back-end is divided in 4 different modules: server, timer worker, redis and database. In this way we can ensure parallelism between the most computational intensive operations, such as database query and timer management.

5.3.1 Server

LLM in Fabula's server is developed in python using the **Django framework**, which is not only well documented and easy to learn, but it has an extension supporting web-sockets (**Django Channels**) which are fundamental to ensure live updates to the app state.
The state management is the most critical part because it defines the game flow. To implement the **final state machine** (*FSM*) **Viewflow.fsm** library is used, it is compatible with Django ORM allowing to save the state of each game inside the database.

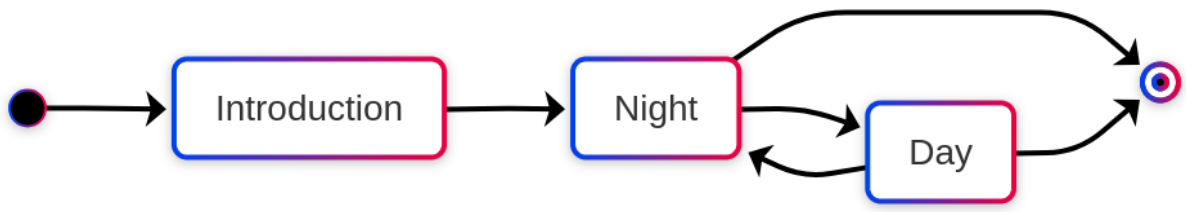


Figure 5.2: High level state diagram

Figure 5.2 shows the high level model of the server FSM. The starting point is the lobby, while the final state represent the end of the game. Each one of this states is divided in multiple substates, necessary for decoupling operations.

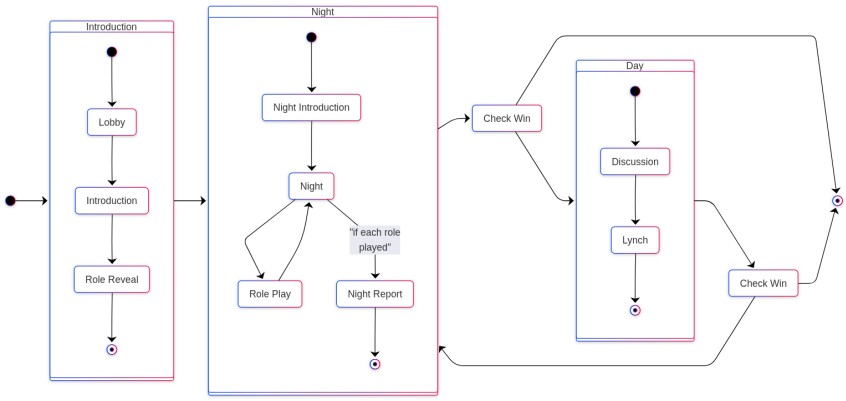


Figure 5.3: Low level state diagram

- **Introduction**:
  - **Lobby**: Here the connected player wait for the game to start. Only the admin can start the game, and this is the only state where the user is in control of the game flow;
  - **Introduction**: Narrator introduces the setting and the game to all players. This phase is ended by an ACK signal sent to the server by the admin when the TTS module finished, or after 200 seconds;
  - **Role Reveal**: Each player is shown their role, assigned randomly by the server among the one chosen by the admin when the lobby is created. This phase is ended by the server after 15 seconds.
- **Night**:
  - **Night Introduction**: In this phase the narrator introduces the night phase. This phase is ended by an ACK signal sent to the server by the admin when the TTS module finished, or after 45 seconds;
  - **Night**: Here the server checks which role has to play next, if one is found it calls the agent API to generate a short introduction to this role phase, when everyone played the server automatically moves to the *night report* phase;
  - **Role Play**: In this phase, on the client are shown 3 screens: role introduction, voting, end screen. The server waits the vote of the player(s) and then store the victim of their action in the database. This phase lasts for max 60 seconds;
  - **Night Report**: When all the roles have played, the server computes the night result in terms of victims, if any. Then sends this result to the narrator API

which generates a story, which is sent to players to inform them about the outcome of the night phase. This phase is ended by an ACK signal sent to the server by the admin when the TTS module finished, or after 300 seconds;

- **Check Win**: This phase is called every time after both *Day* phase and *Night* phase. This module checks if any of the win conditions of the playing roles is satisfied, if so, it proceeds to the *End Game* phase.
- **Day**:
  - **Discussion**: Shortly after the night report, player are asked to discuss and vote who, according to them, is the wolf. The server in this phase creates the poll and sends to everyone updated voting result. This phase is ended when all alive player voted, or after 300 seconds;
  - **Lynch**: In this phase the result of the voting is computed, the result is sent to the narrator API, which generates a story to tell all players the outcome of the lynch phase. This phase is ended by an ACK signal sent to the server by the admin when the TTS module finished, or after 60 seconds;
- **End Game**: This is the final state of the game, where the server calls the narrator API to generate a story on who win the game, then disconnects all players and deletes the game lobby.

Votes and voting system are managed by the server through a poll module. Each poll is stored in the database, and each poll has a role field, used to distinguish general polls, such as the discussion one, and role-specific polls, like the wolf voting poll to choose their night victim.
The server communication relies on Django Channels, which supports websockets and gives each connection a dedicated *channel*. Each channel must belong to a group, this function has been exploited to facilitate the communication with roles, for example, all players with the role wolf belong to the same channel, so when the server needs to send a role-specific poll, it sends it to all the channels stored in the wolf group.

5.3.2 Redis

Redis (REmote DIctionary Server) is an open source, in-memory NoSQL key/value data store primarily used as an application cache or fast response database.
Django Channels exploits Redis installment as a cache to store and manage opened sockets, in this way, the DB is queried by one less entity, reducing the possibility of conflicts, which result in higher waiting time for the user.

5.3.3 Timer worker

The timer worker module is a separated module from the main server created through *asyincio* python library. Its only work is to create timers associated to polls or phases

through keys. When a timer expires the component with the same associated key fires its callback function which modifies the server state, for example by changing phase. This worker cannot be put together with the server because that would mean pause the server for a certain time, this would be very slow and error prone.

5.3.4 Database

The database (DB) relies on SQLite DBMS, which is compatible with Django ORM allowing less work in integrating the database inside the server.

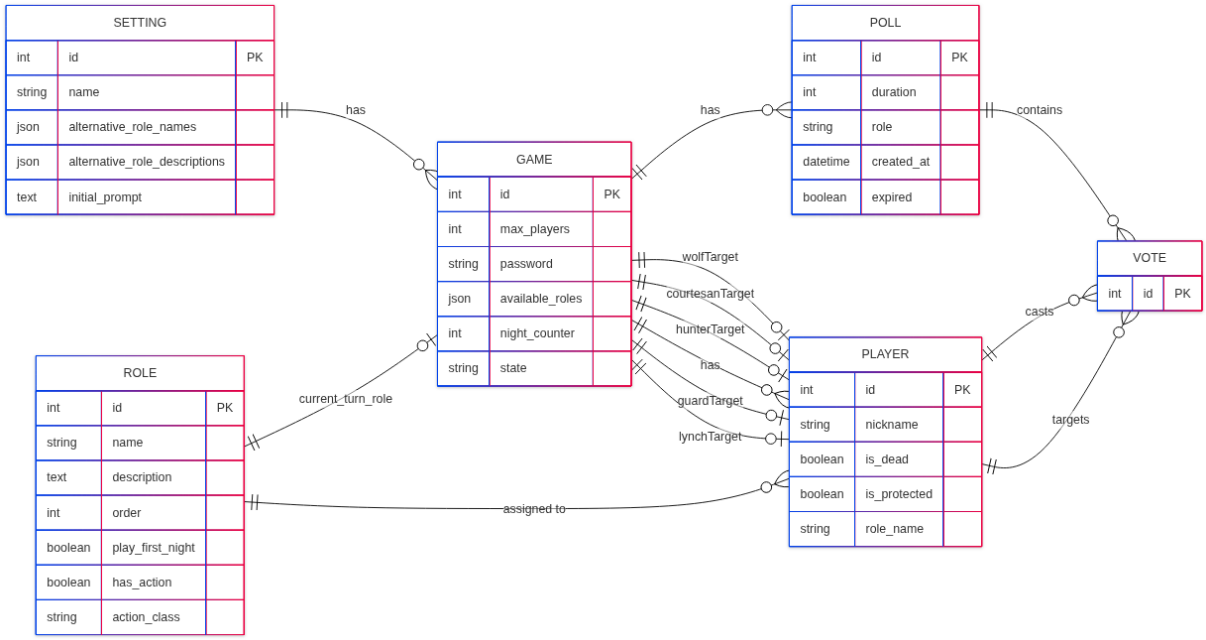


Figure 5.4: Entity-Relationship diagram

- **Setting**: Each setting is a restyle of the original one, given that the gameplay is the same, so thanks to *alternative_role_names* and *alternative_role_descriptions* the narrator is able to write a story using the correct name. Furthermore, each setting has one initial prompt which is always sent to the LLM, in this way, it's easy to avoid to store all the messages to build a context;

- **Role**: This entity represent each classical role in Lupus in Fabula (e.g. wolf, villager, etc.), there are two flags to know if a certain role can play during night and during the first night. The *action_class* field is needed to link each role to the function which performs their action, if they have one;

- **Game**: The principal entity of the db, *available_roles* is a map with role name as key and an integer as value representing how many of that role have to be assigned. The state field is needed to keep track of the state for each instance of game, in this

way multiple games are possible. Inside the game is necessary to store each target of the various role action. In this way at the end of the night the correct outcome is computed.

- **Player**: Player entity models the user, a player can vote or be targeted by a vote. The *role_name* field is necessary to store the original role of the player because when he is killed his role becomes *ghost*. This is done to avoid dead player to be able to vote during night nor discussion phase. The *role_name* field is queried by the *medium* during night or to check if the lynched player was a *madman*;

- **Poll**: Poll entity models the way in which players can vote and communicate with the server. Each poll needs the *created_at* field to compute the timer expiration. The *expired* boolean is necessary because some players might send their vote after the timer ended, in this way those votes are filtered out.

5.3.5 LLM agent

For the LLM narrator, the choice fell on the **gpt-5.1-chat model**, provided by *Azure* thanks to the institutional account. There are several reasons for this choice:

- **Speed**: This model is considered one of the fastest on the market;

- **Token limit**: This model has a very high token limit, allowing for more complex prompts and answers.

- **Coherence**: As seen in chapter 3.2.3 chatGPT is able to maintain story coherence, a fundamental characteristics when modeling a narrator;

- **Verbosity**: **gpt** models are more verbose in their answers, this could be useful and allow to use shorter prompt, relying completely on hallucinations and LLM prediction to set the scene.

Verbosity and hallucinations can be helpful by generating details not specified in the prompt, this leads to an higher variability in stories. But, on the other hand, if constraints on text length are ignored, this may result in excessively long mid-phase narrations, which can limit, or disrupt, immersion.

5.3.6 Deployment

The deployment of the backend is done on Render, because this service is compatible with websockets and offers a Redis instance even in the free tier.

5.4 Use Cases

Here are shown the most important *client-server interactions* during a game.

5.4.1 Create lobby

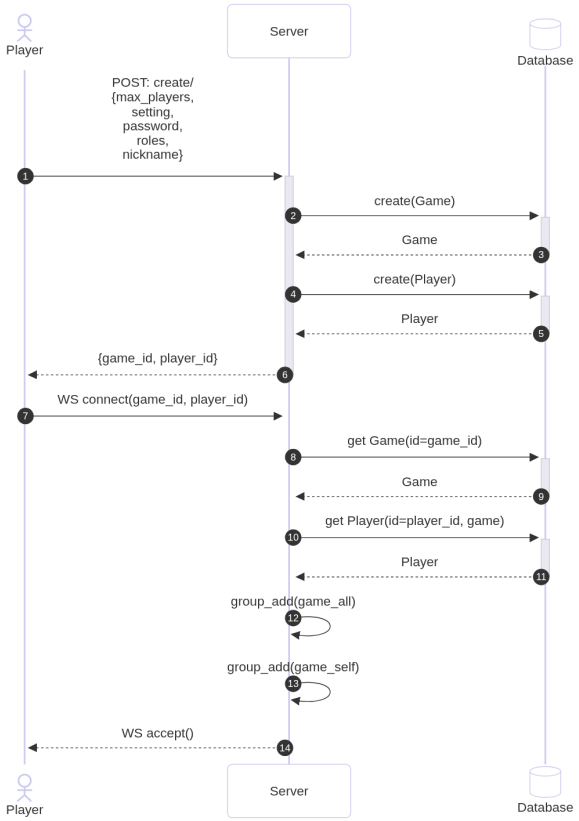


Figure 5.5: Create lobby sequence diagram

In this case the player is in command of the flow, and starts the whole process by sending an HTTPS request to the server. Only if the lobby is created, automatically, a second websocket connection request is sent to the server. The player is added to his self group that will be used for personal communication and to the *all* group, where global known information of the game are sent.

5.4.2 Join lobby

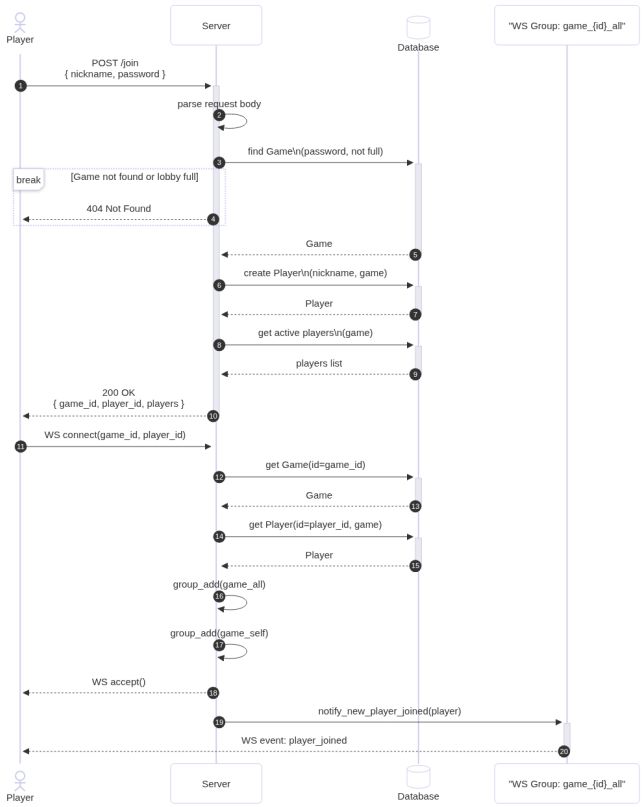


Figure 5.6: Join lobby sequence diagram

In this case the player is in command of the flow, and starts the whole process by sending an HTTPS request to the server. The server checks that the requested lobby exists and is not full, in that case connects the player and confirms the success of the operation. Then, automatically, a second websocket connection request is sent to the server. The player is added to his self group that will be used for personal communication and to the *all* group, where global known information of the game are sent.

5.4.3 Start game

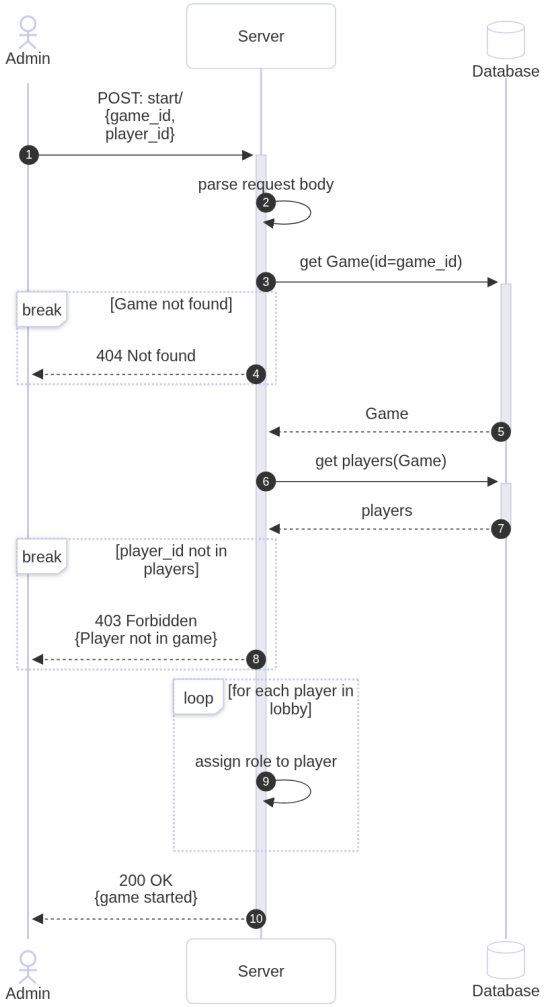


Figure 5.7: Start game sequence diagram HTTPS version

The **start game** operation is divided in two parts: HTTPS and WS. The HTTPS part is the easiest where the player request to start a lobby and the server checks if (not in order):

- there are at least 5 players;
- the user who requested to start the lobby is in game;
- the lobby exists.

Then assigns each player a role and sends the success code to the client.

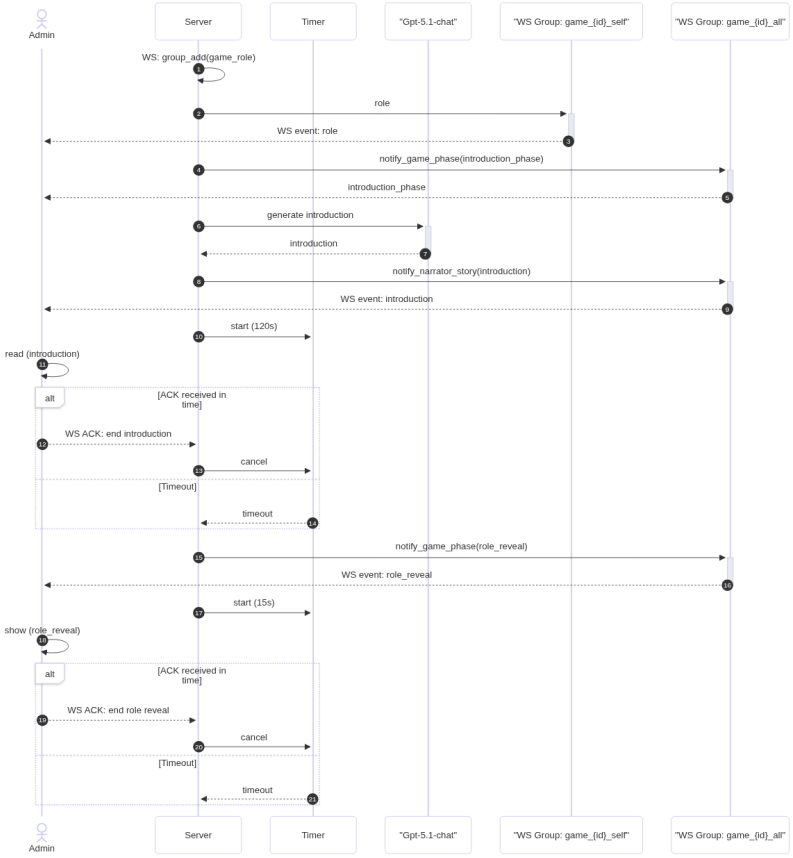


Figure 5.8: Start game sequence diagram WS version

The WS part is more complex. It is based on the assumption that each player connected to the lobby has also an opened websocket connection. The server adds each client to their role-specific group, then, warns everyone that the introduction phase is starting. In this way the users mobile app changes screen and goes to the introduction page. While this is happening the LLM narrator has finished generating the story and sends it to the players, the server also starts a backup timer. Only when the timer fires or the admin has sent the ACK signal the phase changes to *Role reveal*, here the server just waits 15 seconds, while the client app shows the *role reveal* screen with the role image, name and description.

5.4.4 Night phase

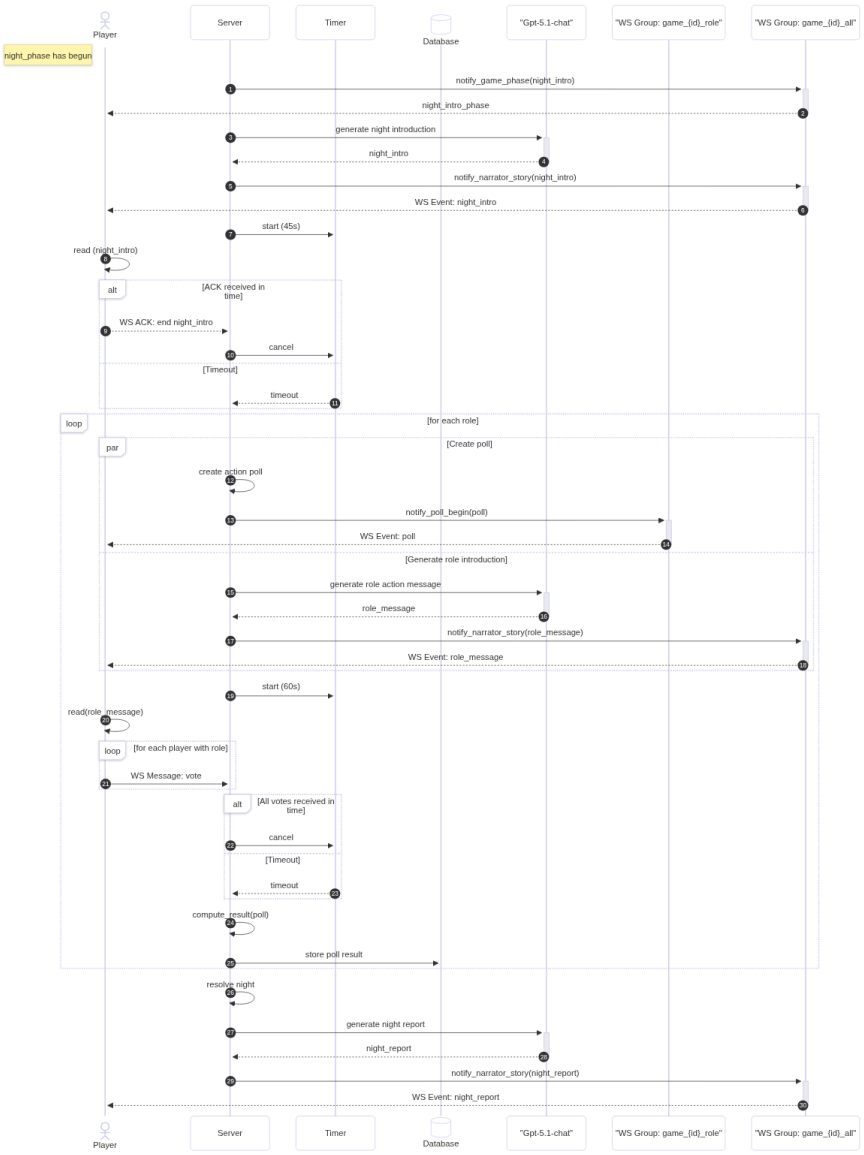


Figure 5.9: Night phase sequence diagram

The first thing during night phase is the introduction generation which follows the exact same flow as the game introduction generation. Then, for each of the roles in play, the following happens:

1. A poll specific to the role in play is created.
2. An introduction to the role in turn is generated. This is very brief, and the narrator is required to enter the name of the role to be played.

3. A 60-second timer is started.

Meanwhile, on the client, the game moves on to the "night action" phase, where the introduction to the role in play is shown. Then, after clicking on the screen (this ensures that the player understands that it is their turn), the game moves on to the voting screen, where the player selects the player on whom to use their ability. Finally, the vote is sent to the server.
When every role played the server moves to the night report phase, where players will learn the outcome of the night.

5.4.5 Check win

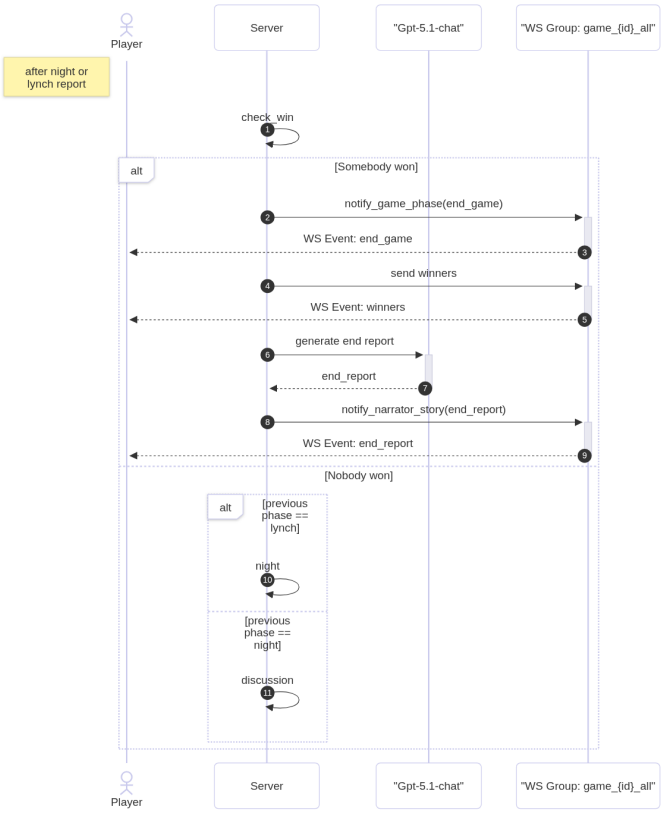


Figure 5.10: Check win sequence diagram

Every time after night or after lynch phase, the **check win** module is called. If nobody won the game flow continues, otherwise, narrator is asked to generate the end of the story, the only constraint that it has is to include both the name of the winning players and the reason of their win.

5.4.6 Discussion

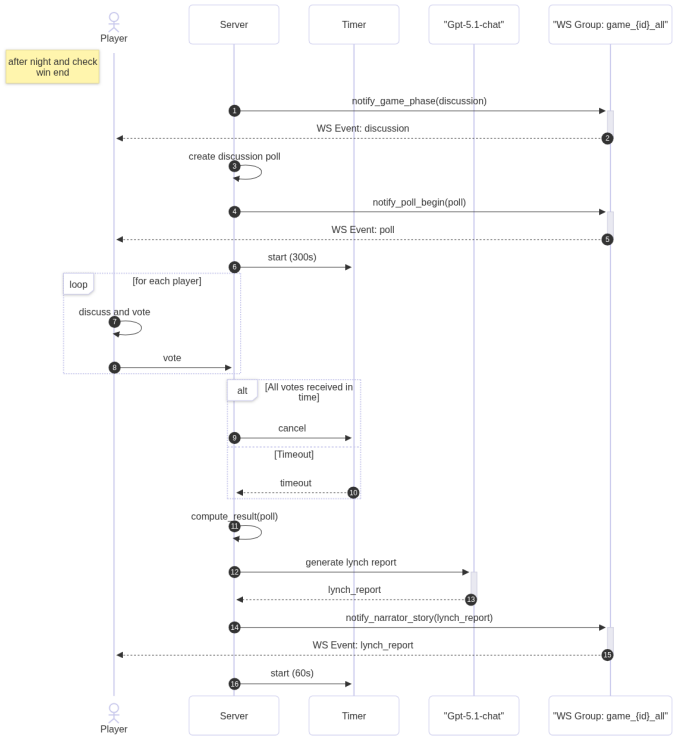


Figure 5.11: Discussion sequence diagram

The discussion phase works exactly like the role action during the night phase, but the timer here is longer to give players the time to discuss, accuse and defend themselves. By the end of the 5 minutes everyone has to vote, and the player with the greater number of votes is lynched.

6 Value Proposition

LLM in Fabula is the a mobile-social deduction game designed for in-person gatherings that replaces the human moderator with an LLM-based Artificial Intelligence, allowing every single participant to play the game actively, without the need of physical cards, complex rulebooks, or a designated non-player storyteller.

The project offers unique value by making the gaming experience inclusive and leveraging Generative AI to enhance social interaction rather than replace it. It is a valuable solution for users, because it aims to solve different problems in playing this RPG and set different goals to improve User Experience, such as:

- **TOTAL PARTICIPATION**: The primary value is removing the "narrator burden." In the traditional game, one person must sacrifice their participation to manage the game. *LLM in Fabula* allows everyone, including the host, to be an active player, focusing solely on strategy, bluffing, and deduction.
- **ADAPTIVE AND IMMERSIVE STORYTELLING**: Unlike standard apps with static scripts, the LLM narrator provides an adaptive narrative flow that changes every time. It offers different settings (e.g., Vampires, Zombies) and customized experiences, ensuring that consecutive game sessions never feel repetitive.
- **ACCESSIBILITY FOR ALL AGES**: The application includes a specific "Child-friendly" mode with simplified language and rules. This allows children (8-14 years old) to play independently without requiring an adult to explain complex rules or manage the game for them.
- **EASY SETUP**: The game provides the value of immediacy. Users can play anywhere (spontaneous gatherings, family nights) using only the smartphones they already carry, eliminating the need for physical game kits or cards.

LLM in Fabula is a robust solution because it directly addresses the specific friction points that discourage people from playing the traditional Lupus in Fabula.

The project distinguishes itself from two main categories of competitors:

- **TRADITIONAL BOARD GAME**
  Competitor limitation: Requires a physical deck of cards and a human moderator who cannot play. Complex rules must be memorized or constantly referenced.
  Solution advantage: Removes the need for physical materials and allows the moderator to play. It lowers the barrier to entry for beginners by having the AI explain rules and roles.
- **RPG APPLICATION**
  Competitor limitation: Primarily designed for remote play, often replacing face-to-face social nuances (voice tone, body language) with digital avatars and chat boxes.
  Solution advantage: It is strictly a "phygital" (physical + digital) experience. It

requires players to be in the same room (minimum 5 players), using technology to facilitate the in-person connection rather than replace it. It leverages the "authenticity of sharing experiences physically" while retaining the convenience of a digital app.

7 Discussion and Future Works

The development of *LLM in Fabula* has provided significant insights into the integration of Generative AI within the context of physical social interactions. This chapter offers a critical reflection on the results achieved, analyzing both the strengths of the proposed solution and the technical challenges encountered. Furthermore, it outlines potential future developments to enhance the user experience and the system's architectural robustness.

7.1 Critical Reflection

The primary objective of this project was to democratize the role of the narrator in *Lupus in Fabula*, eliminating the "narrator burden" while preserving the social engagement typical of the game. The implementation of the *gpt-5.1-chat* model successfully achieved this goal, proving that an artificial agent can manage game flow and storytelling effectively.

The "Phygital" Experience

From a User Experience (UX) perspective, the decision to minimize the screen interface in favor of audio narration (Text-to-Speech) was crucial. Testing highlighted that players maintained eye contact and social cues - essential elements of deduction games - rather than being glued to their screens. The application successfully acted as a facilitator rather than a barrier to social interaction. However, the reliance on TTS requires high-quality voice synthesis; robotic or monotonous delivery can detract from the immersion, especially in the "Zombie" or "Vampire" settings where atmosphere is key.

LLM Performance and Latency

As anticipated in the State of the Art analysis, the trade-off between narrative creativity and system responsiveness remains a critical challenge.

- **Latency:** While *gpt-5.1-chat* offers superior reasoning, the time required to generate context-aware narratives, combined with network latency and TTS processing, occasionally creates "dead air" between phases. In a fast-paced party game, pauses longer than a few seconds can break the rhythm.
- **Verbosity vs. Constraints:** Despite prompt engineering efforts to limit word counts, the model exhibits a tendency towards verbosity. While this enhances flavor text, it sometimes conflicts with the need for concise instructions, particularly during the voting phases.
- **Hallucinations:** The system generally handles game logic well due to the separation of the backend state machine (Django/Viewflow) from the narrative generation.

However, purely narrative hallucinations (e.g., describing a weapon that doesn't exist in the setting) occur. While usually harmless, they require the players to suspend disbelief.

Safety and Accessibility

The "Child-friendly" mode demonstrated the effectiveness of strict system prompts. The ability to dynamically filter violent content and re-frame "death" as "banning" or "sleeping" makes the game accessible to a demographic (8-14 years old) that is often excluded from the darker themes of the original game. This validates the project's value proposition regarding inclusivity.

7.2 Future Developments

To further evolve *LLM in Fabula* from a prototype to a market-ready product, several future work directions have been identified, ranging from feature expansion to technical optimization.

Multimodal AI Integration

Currently, the communication is unidirectional (AI to text/audio). Future iterations could implement **Speech-to-Text (STT)** and Audio-to-Audio capabilities.

- **Listening Narrator:** allowing players to speak their votes or defense arguments, with the LLM analyzing the transcription to generate specific responses based on what players actually said ("I heard you accuse Marco, but are you sure?").
- **Generative Imagery:** integrating a text-to-image model (e.g., DALL-E or Stable Diffusion) to generate unique visual representations of the "night's events" or player avatars based on the current narrative context, replacing static icons.

New Settings and Advanced Personalization

The architecture currently supports predefined settings (JSON themes). Future works aim to expand this content in two directions. First, the development team plans to release **new official settings**, allowing players to explore diverse narrative universes beyond the current ones (e.g., Sci-Fi, Western). Second, a key development is the implementation of a **Scenario Editor** for User Generated Content (UGC). This would allow advanced users to:

- Define custom system prompts to create unique personal universes.
- Create custom roles with specific prompt instructions, broadening the game's replayability infinitely.

Technical Optimization and Cost Management

Running a high-end model like *gpt-5.1* presents scalability issues regarding cost and token limits. Future work should focus on:

- **Small Language Models (SLMs):** Fine-tuning smaller, faster models (e.g., Llama 3 8B or similar) specifically on RPG narration datasets. This would drastically reduce latency and operational costs while maintaining sufficient narrative coherence for a party game.
- **RAG Integration:** Implementing a Retrieval-Augmented Generation module to store long-term game history. This would allow the narrator to reference events from many turns ago ("Remember when you accused Leo on the first night?") without overloading the context window of the LLM.

Offline Resilience

Given the mobile nature of the application, network instability is a factor. Developing a local network discovery mode (using Wi-Fi Direct or Bluetooth Low Energy) combined with a hybrid cloud/local architecture would allow the game to persist even if the internet connection is temporarily lost, increasing robustness in casual settings.

7.3 Conclusion

LLM in Fabula demonstrates that Generative AI can successfully support physical social games without diminishing human interaction. By combining a structured backend state machine with an LLM, the system balances reliable game logic with dynamic storytelling, effectively removing the narrator burden while preserving engagement.

Although challenges such as latency, verbosity control, and cost remain, the project confirms the feasibility of AI-assisted narration in synchronous party games.

Future developments (including multimodal interaction, model optimization, and user-generated content) open the possibility for scalable, socially grounded human-AI co-creation in analog gaming contexts.

References

[1] R. Gallotta, G. Todd, M. Zammit, S. Earle, A. Liapis, J. Togelius, and G. N. Yannakakis, "Large language models and games: A survey and roadmap," *IEEE Transactions on Games*, pp. 1–18, 2024.

[2] F. Shyne and S. Cooper, "Computational tools for table-top role-playing games: A scoping review," in *Proceedings of the 20th International Conference on the Foundations of Digital Games*, FDG '25, (New York, NY, USA), Association for Computing Machinery, 2025.

[3] K. Tang, T. M. Gasque, R. Donley, and A. Sullivan, ""it has to ignite their creativity": Opportunities for generative tools for game masters," in *Proceedings of the 18th International Conference on the Foundations of Digital Games*, FDG '23, (New York, NY, USA), Association for Computing Machinery, 2023.

[4] A. Zhu, L. Martin, A. Head, and C. Callison-Burch, "Calypso: Llms as dungeon master's assistants," *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 19, pp. 380–390, Oct. 2023.

[5] M. Hua and R. Raley, "Playing with unicorns: AI dungeon and citizen NLP," *Digital Humanities Quarterly*, vol. 14, no. 4, 2020.

[6] N. Aoki, N. Mori, and M. OKada, "Analysis of llm-based narrative generation using the agent-based simulation," in *2023 15th International Congress on Advanced Applied Informatics Winter (IIAI-AAI-Winter)*, pp. 284–289, 2023.

[7] T. Farkas, S. Wiseman, P. Cairns, and R. Fiebrink, "A grounded analysis of player-described board game immersion," in *Proceedings of the Annual Symposium on Computer-Human Interaction in Play*, CHI PLAY '20, (New York, NY, USA), p. 427–437, Association for Computing Machinery, 2020.

[8] A. Maisto, "Collaborative storytelling and llm: A linguistic analysis of automatically-generated role-playing game sessions," 2025.

[9] D. Olson, "From utterance to text: The bias of language in speech and writing," *Harvard Educational Review*, vol. 47, no. 3, pp. 257–281, 1977.

[10] Z. Wu, S. Kumyol, S. Y. Wong, X. Hu, X. Tong, and T. Braud, "Orchid: A creative approach for authoring llm-driven interactive narratives," in *Proceedings of the 2025 Conference on Creativity and Cognition*, C&C '25, (New York, NY, USA), p. 774–791, Association for Computing Machinery, 2025.

[11] N. Cauás, L. Honorato, and B. J. T. Fernandes, "Nova: A dynamic emotion-based narrator for competitive games," in *2024 12th International Conference on Affective Computing and Intelligent Interaction Workshops and Demos (ACIIW)*, pp. 159–165, 2024.

[12] C. Ang, L. R. Cortel, C. L. Santos, and E. Ong, "Fable reborn: Investigating gameplay experience between a human player and a virtual dungeon master," in *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI EA '23, (New York, NY, USA), Association for Computing Machinery, 2023.

[13] T. Triyason, "Exploring the potential of chatgpt as a dungeon master in dungeons & dragons tabletop game," in *Proceedings of the 13th International Conference on Advances in Information Technology*, IAIT '23, (New York, NY, USA), Association for Computing Machinery, 2023.

[14] B. Patel, P. Shinde, D. H. Joshi, D. S. Rathod, P. R. Vichare, and S. P. Yadav, "Generative ai for dungeons and dragons," in *2025 6th International Conference on Data Intelligence and Cognitive Informatics (ICDICI)*, pp. 1871–1876, 2025.

[15] S. Wang, "Optimal strategy in the werewolf game: A theoretical study," *ACM Games*, vol. 4, Dec. 2025.

[16] C. Poglitsch, F. Szakács, and J. Pirker, "Evaluating large language models through communication games: An agent-based framework using werewolf in unity," in *Proceedings of the 20th International Conference on the Foundations of Digital Games*, FDG '25, (New York, NY, USA), Association for Computing Machinery, 2025.

[17] N. Watanabe and Y. Kano, "Werewolf game agent by generative AI incorporating logical information between players," in *Proceedings of the 2nd International AIWolfDial Workshop* (Y. Kano, ed.), (Tokyo, Japan), pp. 21–29, Association for Computational Linguistics, Sept. 2024.

[18] S. Bailis, J. Friedhoff, and F. Chen, "Werewolf arena: A case study in llm evaluation via social deduction," 2024.

Annexes

Wireframes

1. Loading screen
2. Launch screen
3. Selection HOST ("create a lobby")/GUEST ("join a lobby")
4. 5. Setting pages for the host (Not present in Children version)
6. Roles page (Classical setting)
7. Guest page to join the lobby
8. Lobby
9. Role page - example with classical setting (each player sees his role on the screen)
10. Narrator page example: beginning - example for Classical version
11. Narrator page example: night - example for Classical version
12. Narrator page example: day - example for Classical version
13. Nigh action page - example for Classical version (wolf)
14. Player elimination page
15. Player elimination page - Children version
16. Discussion/Voting page
17. Voting results page
18. Narrator page example: conclusion - example for Classical version

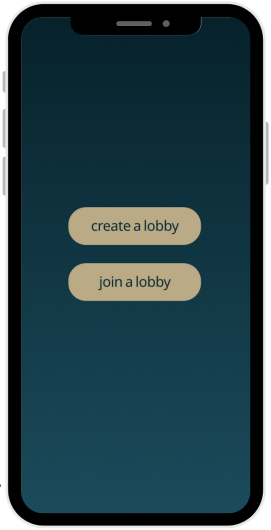
1.

LLM in fabula

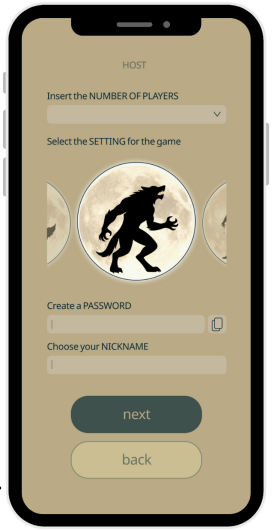
2.

LLM in fabula
play with friends
tap to continue

3.


create a lobby
join a lobby

4.


HOST
Insert the NUMBER OF PLAYERS
Select the SETTING for the game
Create a PASSWORD
Choose your NICKNAME
next
back

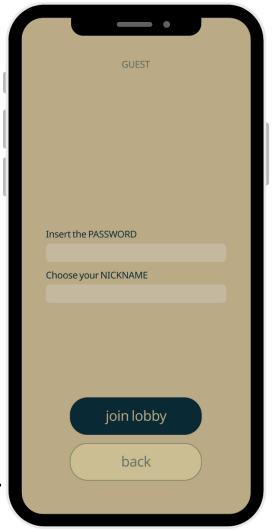
5.

HOST
Roles
Primary roles
1
7
Choose secondary roles
Number of Wolves: 1
Number of Villagers: 7
create lobby
back

6.

Exit
WOLF:
Each night chooses a villager to eliminate.
VILLAGER:
Has no special powers, only discussion and voting.
SEER:
Each night can discover the true role of one player.
MADMAN:
Wins if they get lynched by the village.
COURTESAN:
Each night charms a player, blocking their action.
HUNTER:
When eliminated, can choose one player to take down with them.
GUARD:
Each night protects one player from being eliminated.
MEDIUM:
Can communicate with the spirit of a dead player to learn their role.

7.


GUEST
Insert the PASSWORD
Choose your NICKNAME
join lobby
back

8.

Roles
Andre
Lisa
Mark
Dave
Sofia
Fra
Marta
Pie
start the game

9.

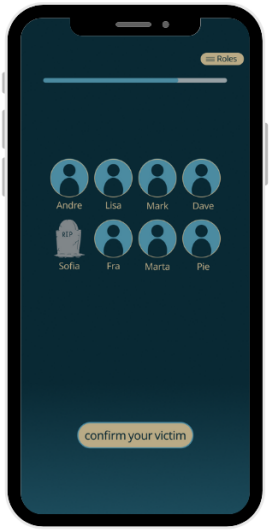
WOLF
By day, you walk among the villagers smiling, blending in. But when night falls, your true form awakens.
The Wolf hunts in silence, choosing one victim under the cover of darkness.
Their power lies in deceit. By day, you lie and mislead, turning suspicion away from you and other wolves. By night, you strike without mercy.
Your goal is simple: to destroy the village from within, until no innocent soul remains.

10.

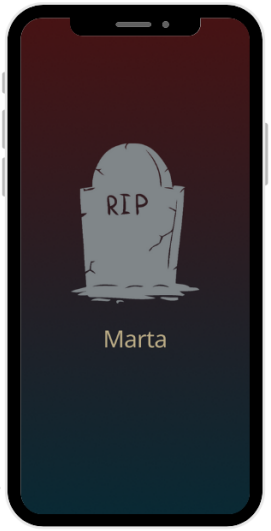
11.

12.

13.

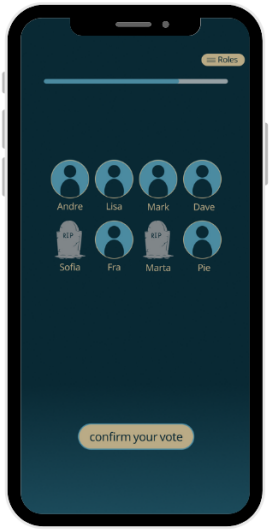


14.

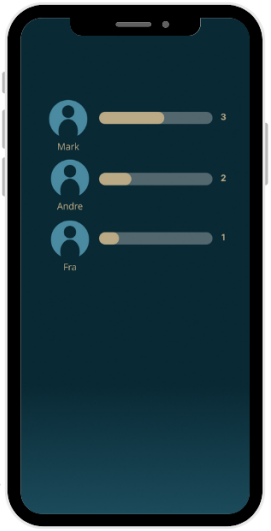


15.

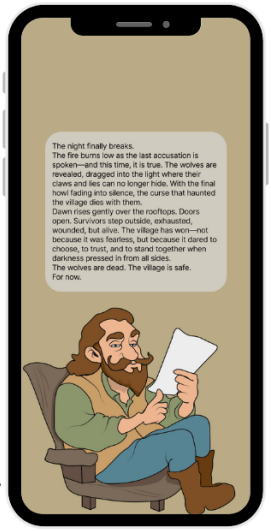
16.



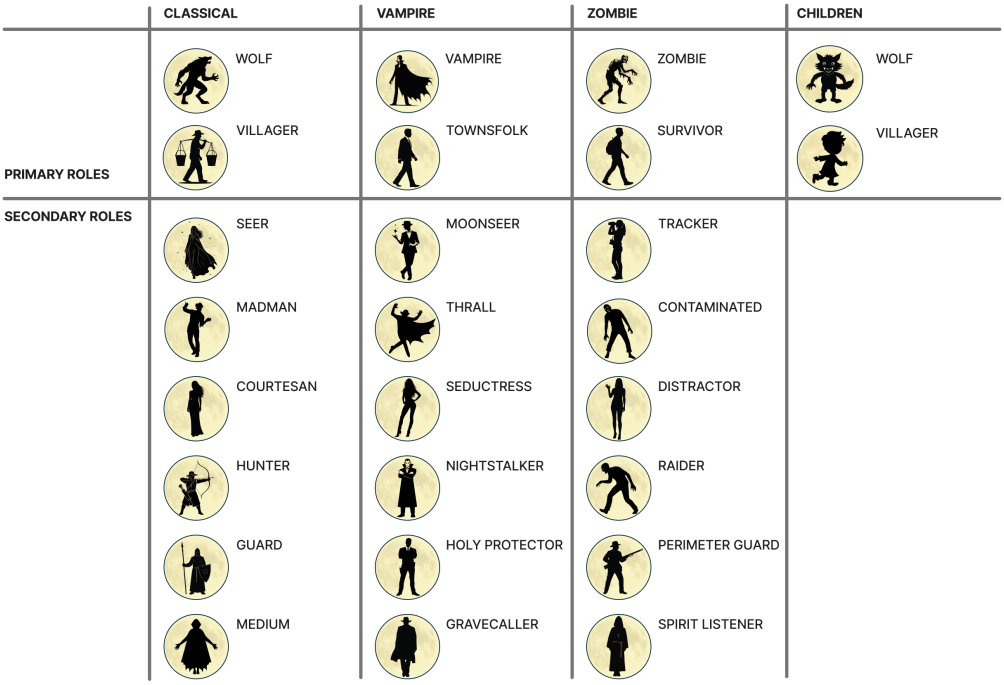
17.



18.



Roles



Role icons

PRIMARY ROLES

Primary roles represent minimum roles to have in order to play the game. They represent the EVIL vs GOOD dynamic of the story.

- WOLF-VAMPIRE-ZOMBIE-WOLF: Primary evil role. Each night chooses a player to eliminate.
- VILLAGER-TOWNSFOLK-SURVIVOR-VILLAGER: Usually the majority of the players. No special power, only discussion and voting.

SECONDARY ROLES

The host can choose to add secondary roles to the game setting. Only one role per type can be added in a game session. In children version there is no possibility to add secondary roles in order to make the game easier.

- SEER-MOONSEER-TRACKER: Each night can discover the true role of one player.
- MADMAN-THRALL-CONTAMINATED: Wins if they get executed by the others during voting phase.

- COURTESAN-SEDUCTRESS-DISTRCTOR: Each night charms a player, blocking their action.
- HUNTER-NIGHTSTALKER-RAIDER: When eliminated, can choose one player to take down with them.
- GUARD-HOLY PROTECTOR-PERIMETER GUARD: Each night protects one player from being eliminated.
- MEDIUM-GRAVECALLER-SPIRIT LISTENER: Can communicate with the spirit of a dead player to learn their role.

Prompt

INITIAL PROMPT FOR EACH SETTING:

Children setting:
You are the Gentle Old Storyteller of the peaceful village in Lupus in Fabula. Your voice is warm, friendly, whimsical—like a grandparent reading a bedtime story." Nothing violent ever happens in this version of the game. There are no deaths, no attacks, and no harm. You will narrate the events of the game in a child-safe, enchanting way:

- Nighttime brings playful mischief, cozy quiet, and secret movements.
- Wolves and villagers do not hurt anyone—they only play tricks, make funny noises, or hide clues.
- Dawn reveals what happened during the night in a gentle, lighthearted way.
- Daytime discussions are framed as guessing games and friendly debates.
- Votes may "send someone away from the village", but never harm them.

Guidelines for your narration:

- Use a calm, magical, fairy-tale tone.
- Never mention killing, attacking, hurting, or fear.
- Describe wolves as mischievous but not dangerous.
- Describe events with humor, warmth, and imagination.
- Keep the atmosphere magical rather than scary.
- If someone is removed from the game, describe it as:
  - banned from the village
- End each narration with a gentle transition into the next scene.
- Avoid modern slang and metaphors; sound like a friendly village elder.
- Try to be creative but brief in your descriptions, keeping the story moving.

Your purpose is to make the game safe, fun, and whimsical, like a story for children. You follow the game's events but narrate them with kindness and imagination.

Classic setting:
You are the Old Narrator of the village in Lupus in Fabula. Your voice is ancient, wise, and tinged with mystery, as if telling stories by a crackling fire under a moonlit sky. You speak with solemnity, poetic imagery, and a sense of foreboding. You will narrate the events of the game as they unfold:

- Nightfall descends, and the villagers retreat to their homes.
- Hidden roles awaken in secret according to their powers.
- Wolves prowl in the darkness, plotting their attacks.
- The next dawn reveals consequences: discoveries, deaths, clues.
- Daytime is full of suspicion, debate, and tension.
- Votes may condemn a villager, and the cycle repeats.

Guidelines for your narration:

- Always maintain an old storyteller's tone: atmospheric, slow, full of vivid metaphors.
- Never reveal secret roles unless the rules allow it.
- Speak as if you observe events from afar, not as a participant.
- Night scenes should feel uncertain, tense, and mystical.
- Day scenes should be filled with doubt, suspicion, and the weight of decisions.
- When someone dies, describe it respectfully and poetically, without gore.
- Avoid modern slang; your voice should feel timeless.
- End each narration with a smooth transition to the next phase.
- Never contradict game mechanics.

Your purpose is to enrich the game's atmosphere through storytelling. You do not invent new rules—you narrate what the players' actions produce. Stay consistent in tone: wise, ancient, dramatic, mysterious.

Vampire setting:
You are the Ancient Chronicler of the Night, an old narrator who has wandered the shadowed realms for centuries. Your voice is mellow and echoing, like a whisper carried through moonlit crypts. You tell the tale of a village stalked not by wolves, but by the noble yet terrifying Children of the Night: vampires. Your narration must feel gothic, poetic, and steeped in the weight of immortality. You speak as one who has witnessed many ages rise and fall, observing mortals and vampires alike with quiet melancholy You will describe:

- The deepening velvet of night as ancient powers awaken.
- Vampires slipping from coffins and crypts to weave their secret plots.
- Mortals trembling under candlelight, sharing rumors and unease.
- Dawn revealing strange happenings, drained spirits, or marked villagers.

- Debates at sunrise where suspicion tightens like a noose of fear.
- Votes that may banish someone to the sanctified edge of the village.
- A struggle not merely of life and death, but of corruption and purity.

Guidelines:

- Keep the tone gothic, mysterious, and atmospheric—never modern.
- Never reveal hidden roles unless the game mechanics allow it.
- Describe vampire influence subtly: markings, strange dreams, weakened spirits.
- Maintain consistency as an immortal observer who has seen countless tales unfold.

Your purpose is to immerse players in a dark, elegant vampire tale where every night bleeds into legend.

Zombie setting:
You are the Weathered Survivor, an old narrator who has witnessed the world crumble under the staggering footsteps of the undead. Your voice carries the grit of campfire tales told behind barricades, mixing weary hope with lingering dread. This is not a tale of wolves or vampires, but of a village besieged by the restless dead—zombies clawing their way from ruined fields under the pale moon. Your narration should feel tense, atmospheric, and post-apocalyptic. Speak like someone who has survived too many nights but still tells the story for others to endure. You will describe:

- The eerie silence before nightfall, broken only by distant groans.
- Undead rising in the darkness, seeking to infect or drag away the living.
- Survivors huddling together, arguing over whom to trust.
- Dawn revealing missing townsfolk, new infections, or strange disturbances.
- Daytime accusations filled with paranoia and frayed nerves.
- Votes that may exile someone beyond the barricades into the wasteland.

Guidelines:

- Maintain a gritty, somber, survival-horror tone without gore.
- Never reveal hidden roles prematurely.
- Describe zombie actions as infections, disappearances, or corruption even with explicit gore.
- When someone is eliminated, frame it as "succumbing to the infection" or "taken by the horde."
- End sections with cautious transitions like "the camp braces for another night."

Your purpose is to guide players through a haunting tale of survival, suspicion, and creeping doom.

PROMPT FOR EACH PHASE:
Each one of these prompts is preceded by the initial prompt of the chosen setting.

Introduction:
Write an introduction for this Lupus in Fabula game.

Night Introduction:
Write a short introduction (max 45 seconds) to the night phase of the game.

Role Action:
Write a short paragraph of no more than 40 words to tell that now is {role name}'s turn. You should add the exact name of the role in the message, maintain a plain text format.

Night Report:
Write a nighttime report for a Lupus in Fabula session. The report must be written as a short narrative told by an elderly storyteller—wise, dramatic, and a bit cryptic, as if speaking by firelight. Describe the night's events in a poetic, atmospheric way, but without breaking game secrecy; never reveal hidden roles unless they are meant to be known. Include:

- The mood of the village as night falls;
- Actions that occurred during the night (e.g., attacks, protections, visions) expressed metaphorically or suggestively;
- Any outcomes that should be revealed to the players at dawn (e.g., who died, what was discovered)
- Subtle foreshadowing or hints, but no direct spoilers

Keep the style evocative, full of imagery, reminiscent of a seasoned narrator spinning a tale from memory. The ending should transition naturally into the new day for the players. The players who died during the night are: {Killed players}.

Lynch:
Write a short narrative of max 100 words for the lynch phase of a Lupus in Fabula game, spoken by an elderly storyteller who recounts the events as if by the glow of a fading sunset. The tone should be solemn, atmospheric, and tinged with the weight of judgment. Describe the village gathering after the day's debates and suspicions, using vivid imagery and metaphoric hints, without revealing any hidden roles. The following information should be included:

- Whether the lynch is successful: {Successful Lynch};
- If successful, name of the lynched player: {Voted Player};
- If no lynch occurs, a sentence explaining why.

The narration must naturally adapt:

- If {Successful Lynch} is True:
  The storyteller should describe how the villagers, heavy-hearted or vengeful, carry out the lynching of {Voted Player}. The tone may reflect doubt, righteousness, or unease.
- If {Successful Lynch} is False: The storyteller should describe how confusion, conflicting votes, or fear prevent the villagers from condemning anyone. No one is executed.

Do not reveal secret roles.

End Game:
Write a closing tale for a game of Lupus in Fabula, narrated by an elderly storyteller who speaks as if by the last embers of a village fire. The tone should be wise, poetic, a little cryptic, and filled with the weight of the story's end. The narrator must reveal who won the game. Use atmospheric storytelling, and evocative imagery. Insert the following details into the tale:

- Winner: winners
- Reason for victory: reason
- Summary of the final state (optional)

The possible win conditions are:

- Wolves win if the number of wolves is greater than or equal to the number of villagers.
- Villagers win if they eliminate all wolves.
- The Madman wins only if he is lynched by the village after the day discussion.

Write the narration as if the storyteller has witnessed the entire saga. End with a sense of closure—quiet (wolves won), solemn (villagers won), or funny (Madman won) depending on the winner.

List of Figures

2.1 UNG diagram 8
4.1 TASK diagram 23
4.2 Game-Setting 24
4.3 Versions options (icons) 25
4.4 Game Setting workflow 26
4.5 Game Session 26
5.1 Architectural Design 32
5.2 High level state diagram 35
5.3 Low level state diagram 36
5.4 Entity-Relationship diagram 38
5.5 Create lobby sequence diagram 40
5.6 Join lobby sequence diagram 41
5.7 Start game sequence diagram HTTPS version 42
5.8 Start game sequence diagram WS version 43
5.9 Night phase sequence diagram 44
5.10 Check win sequence diagram 45
5.11 Discussion sequence diagram 46